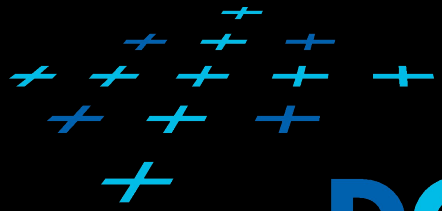
VIZ 06

VISUALIZATION OF VOLUMETRIC DATA

LADISLAV ČMOLÍK and PAVEL SLAVÍK

Department of Computer Graphics and Interaction
Faculty of Electrical Engineering at CTU in Prague



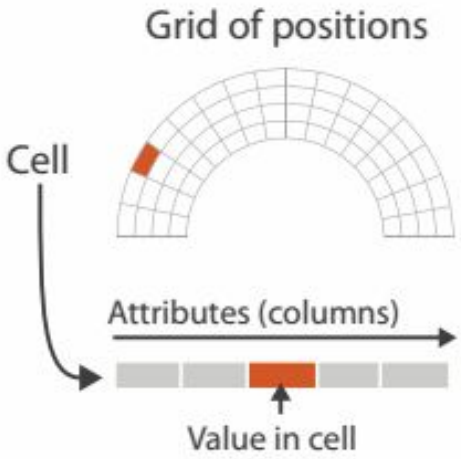
DCGI

Outline

1. Volumetric data
2. Visualization of volumetric data with geometry
3. Direct volume rendering
    1. Ray function
    2. Ray integration
4. Quality of volume visualization
5. Direct volume rendering algorithms
6. Special problems in volume rendering

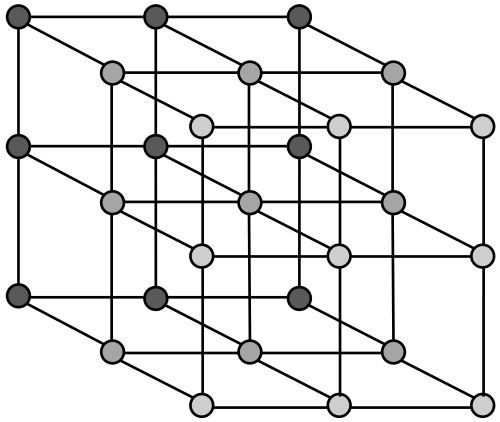
Volumetric data

- Volumetric data are **spatial fields**
  - **3D scalar field**
- Grid is 3D and it is in 3D space
- Grid consists of
  - Positions (vertices) and
  - Cells
- Attributes
  - Both positions and cells may contain values of several different attributes
  - Temperature, pressure, …
- We will consider grids with attributes in the vertices

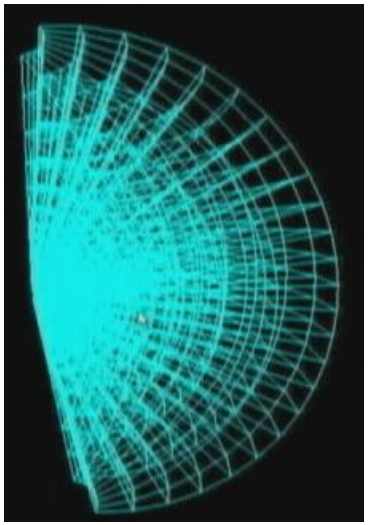


Volumetric data

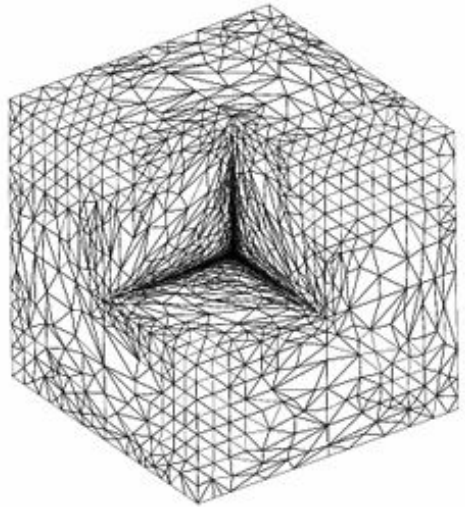
- Discretely sampled on regular or uniform grid in 3D space or generally sampled
  - Regular or uniform grid
    - 3D array of samples
  - Curvilinear grid
  - Tetrahedral grid



Regular



Curvilinear



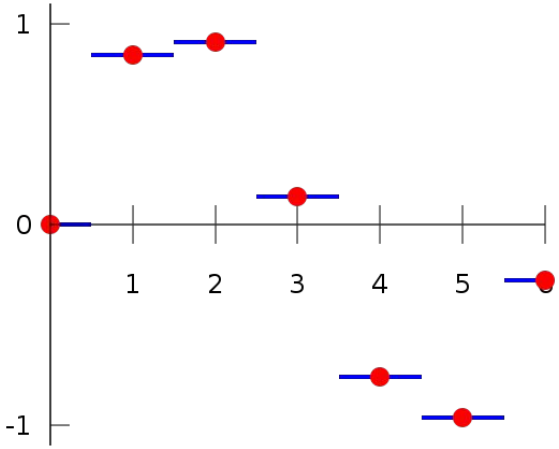
Tetrahedral

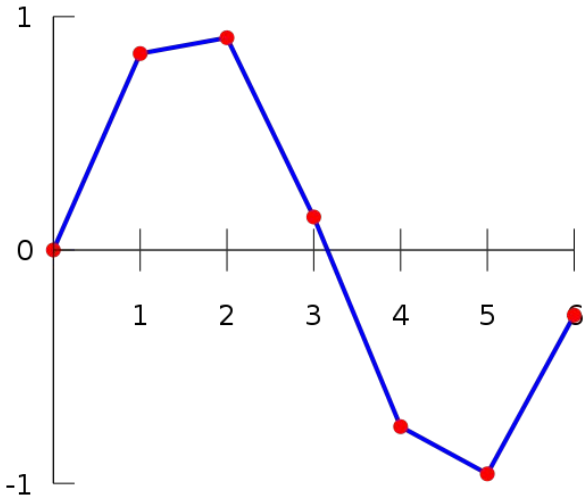
Volumetric data: problems

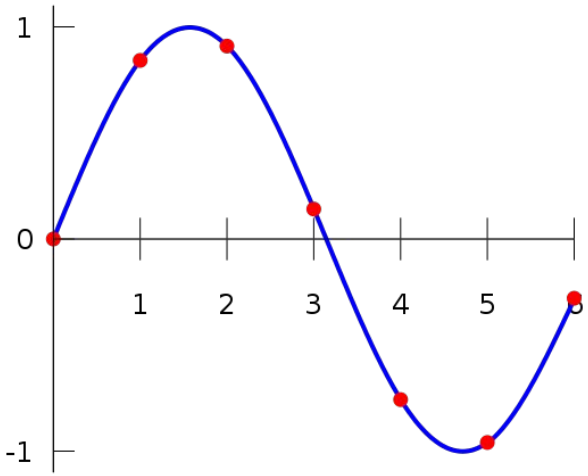
- Big data
- In B-rep 1 million of triangles is a big data
- In volumetric visualization 1 million of voxels is relatively a small data set
  - Regular grid of size 100 x 100 x 100 = 1 milion voxels
  - Regular grid of size 1000 x 1000 x 1000 = 1 bilion (1000 milion) voxels
- Large memory requirements
  - 1000 x 1000 x 1000 x 4B (Integer) = 4 GB
- We want the visualization to be interactive

Data enrichment

- We need to reconstruct the continuous field
- The field can be reconstructed with interpolation
  - Nearest neighbor
    - Value in every point of space correspond to the value of the nearest sample
  - Linear interpolation
  - Cubic interpolation

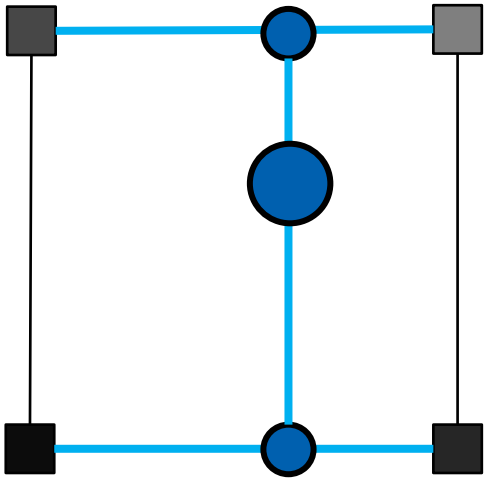


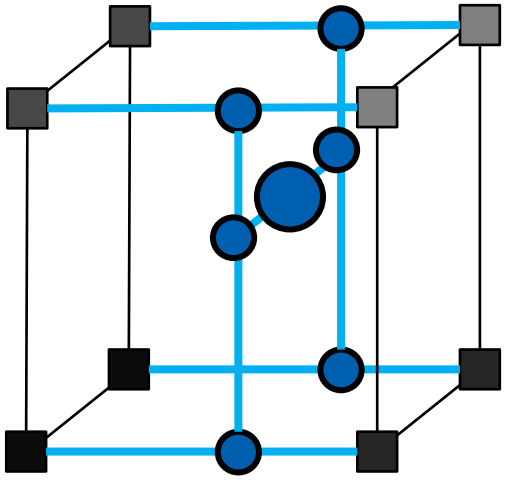




Trilinear interpolation

- Extends bilinear interpolation by another dimension
  - Interpolation from 8 samples
  - 7x linear interpolation

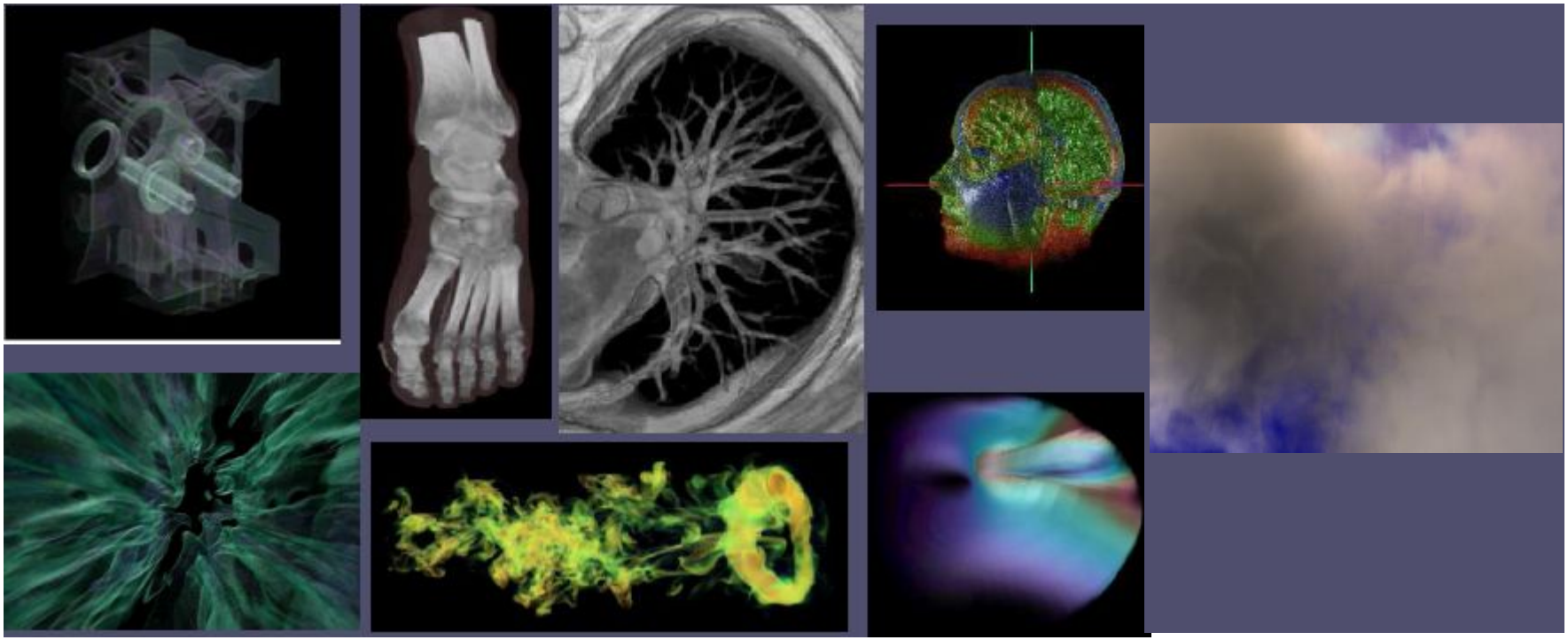




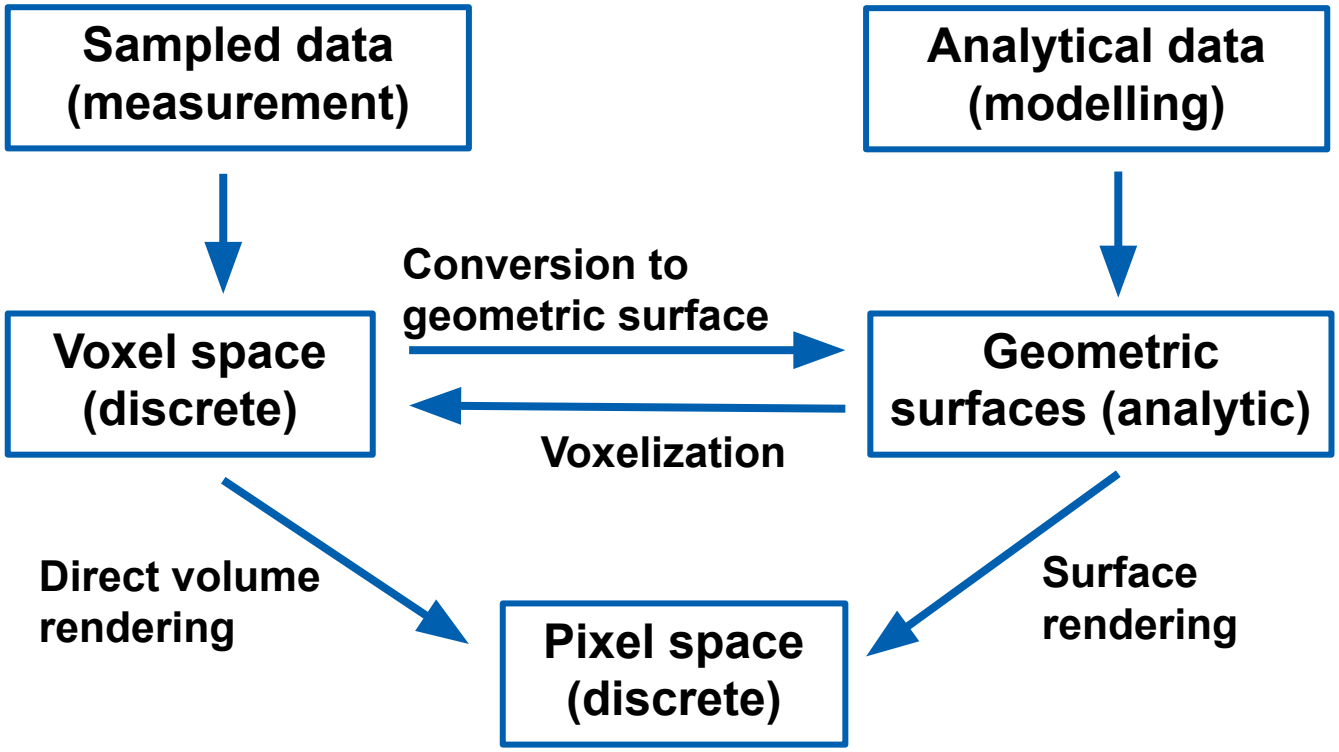
Volume Data Sources

- Measured
  - CT-scan, MRI
  - Seismic data
  - Atmospheric data
- Scientific modeling
  - Computational Fluid Dynamics (CFD) Simulation
  - Plasma physics
  - Atmospheric modeling
  - Geophysical earth models
- Conversion from surface (B-rep)
  - Possible in real-time
- Plenty of others

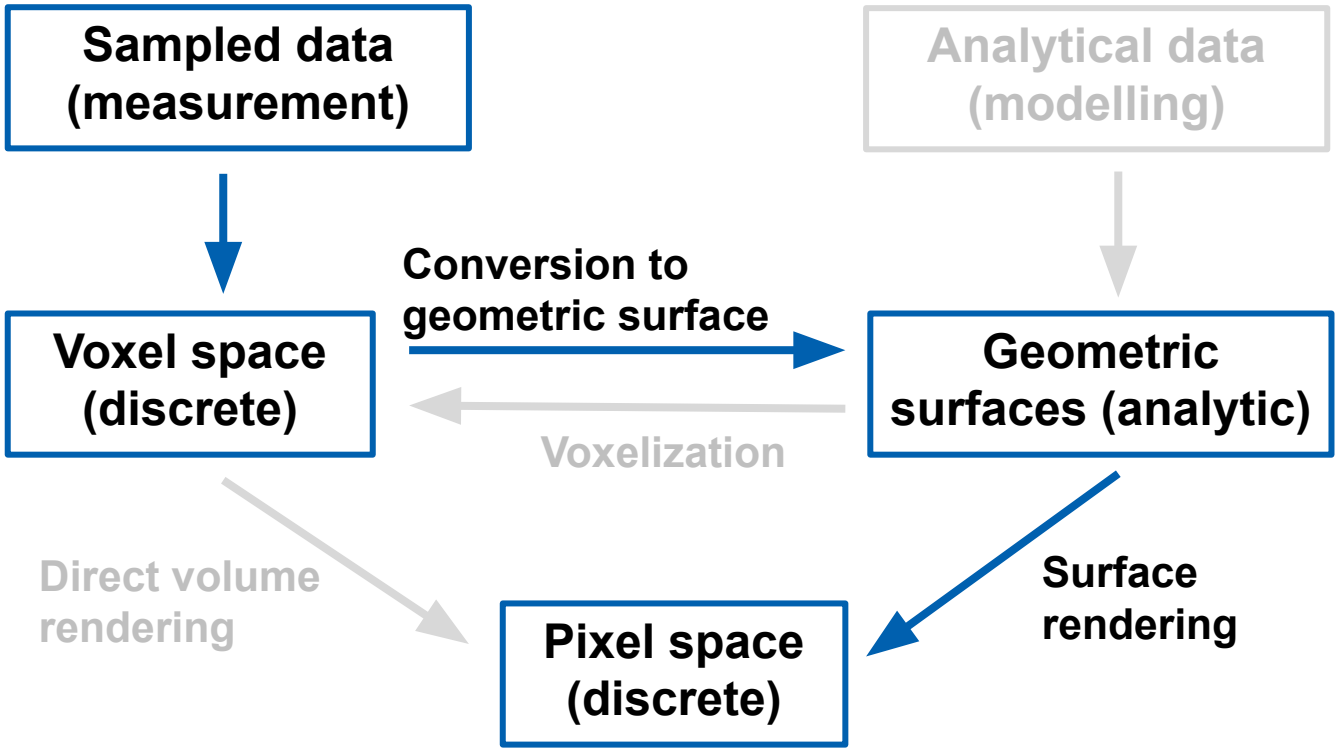
Volumetric Data



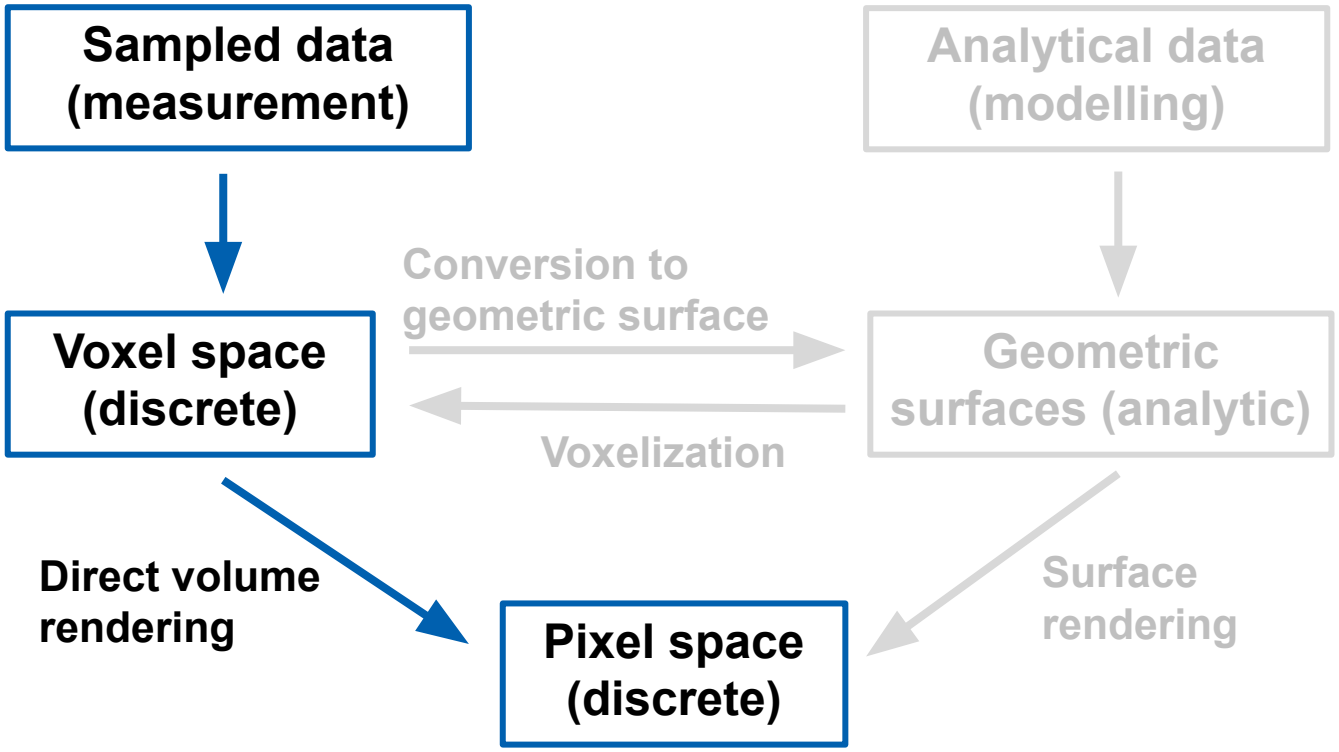
Basic concepts



Indirect volume visualization



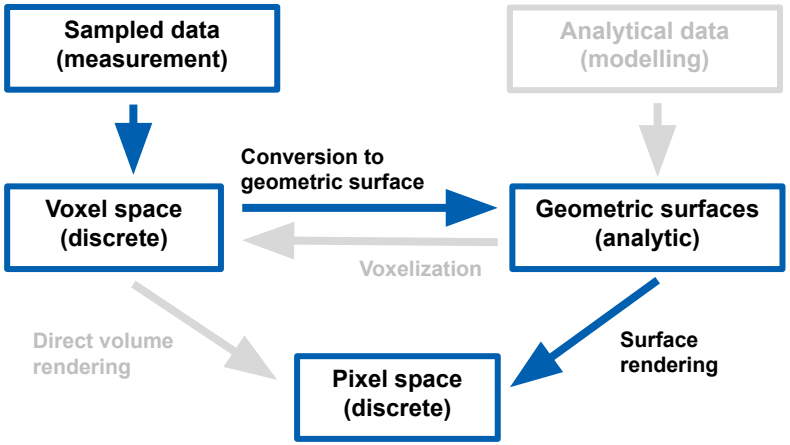
Direct Volume Rendering



Indirect vs. Direct Volume rendering

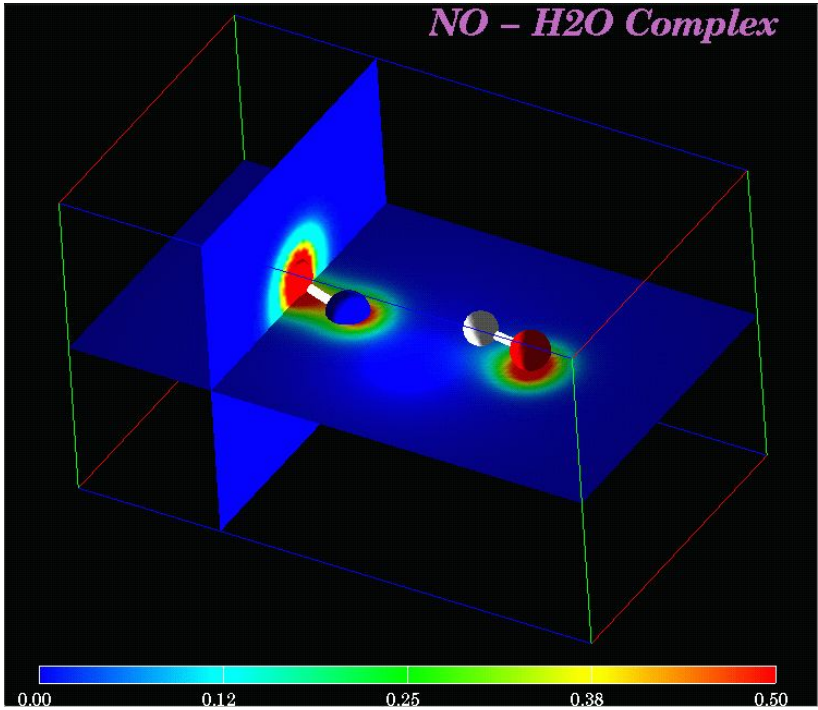
- **Indirect volume visualization**
  - Intermediate geometric representation
    - Cutting plane, tiny cubes, iso-surface
  - *Pros:* Easy shading, perception of shape, fast rendering – can be divided into conversion to geometry and rendering
  - *Cons:* Occlusion, no context
- **Direct Volume Rendering**
  - No intermediate geometric representation
    - Ray function, Ray integration
  - *Pros:* Illustrate the interior, semi-transparency, great flexibility
  - *Cons:* High computational cost, large memory requirements

INDIRECT VISUALIZATION OF VOLUMETRIC DATA



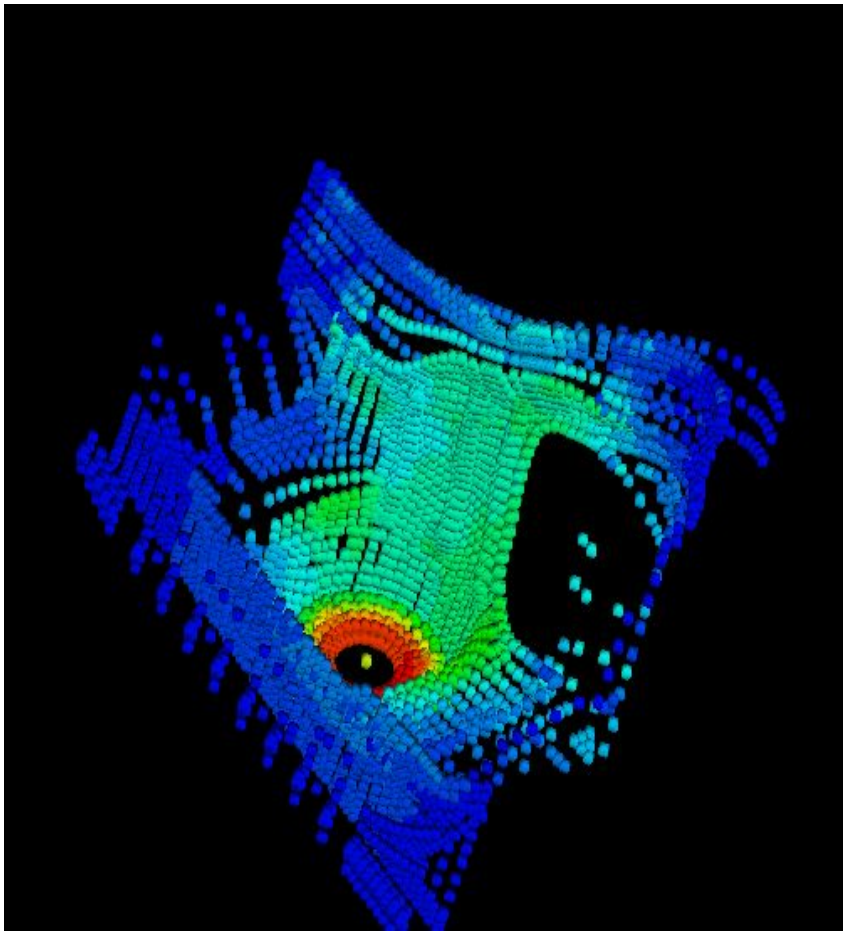
Slicing Planes: 3D problem converted into 2D

- Slicing filters data on the slicing plane
- We map the data on the slicing plane to color
  - As discussed in the previous lecture
- We can visualize data on several slicing planes



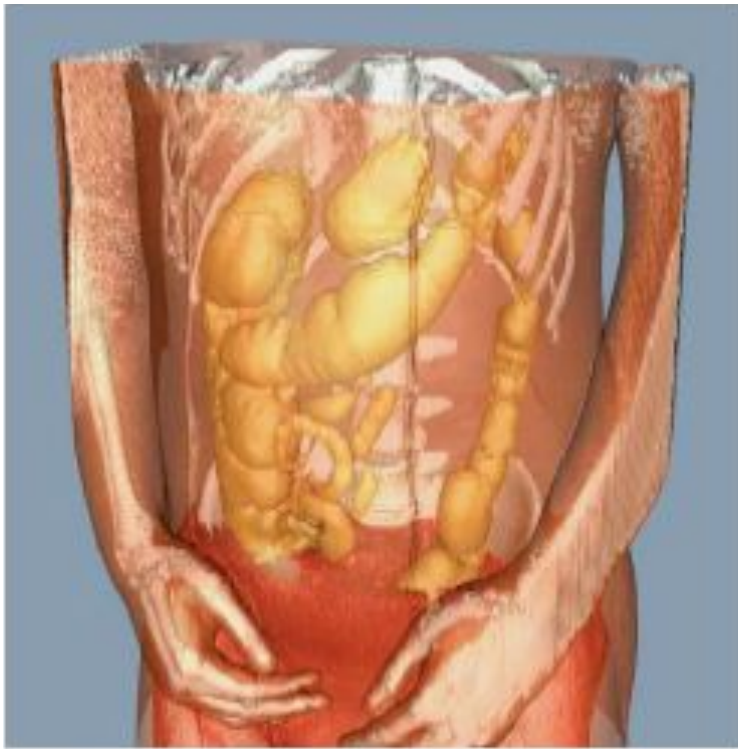
Tiny Cubes

- Place a collection of small cubes (or spheres) in the volume
  - We filter unimportant cubes otherwise they would occlude the cubes of interest
- Define color for each cube
- Project the cubes on the image plane

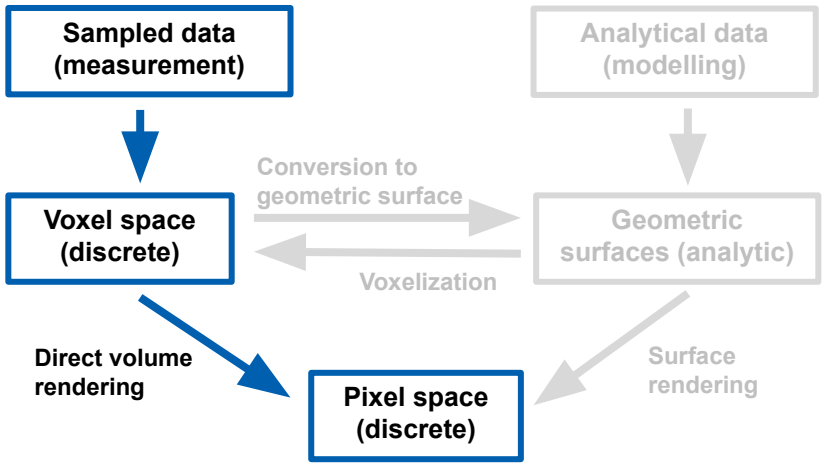


Contouring

- Iso-surface calculation
  - Marching cubes
  - Marching tetrahedrons
- Surface rendering

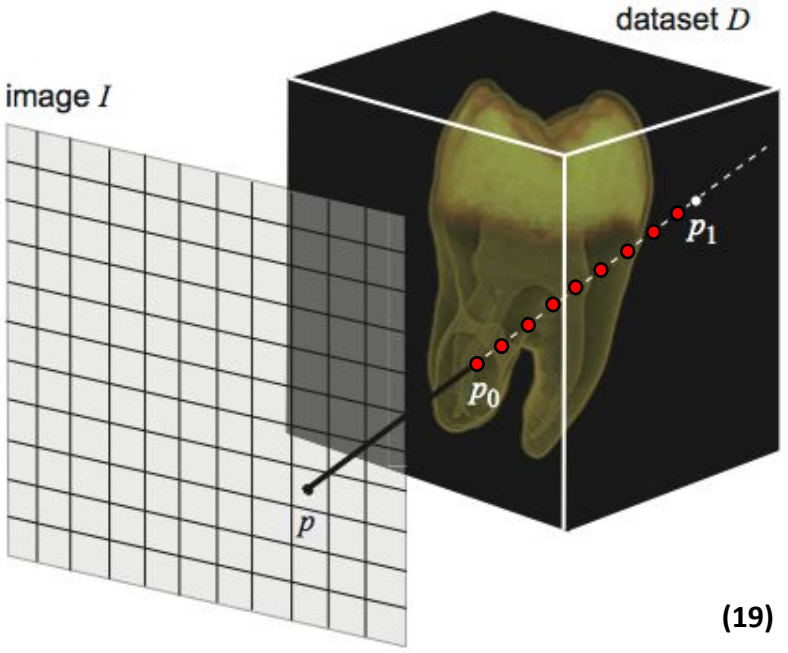


DIRECT VOLUME RENDERING



Ray Marching

- Direct volume rendering (DVR) can be considered as mapping of 3D data onto 2D image
- Ray is constructed for each pixel of the image
  - Intersections with the data block are calculated
  - We march in small steps along the ray starting from one of the intersections until we reach the other intersection
    - Front to back
    - Back to front
  - We do this for all pixels (in parallel)
- Two ways how to map data on the ray to color of the pixel
  - Ray function
  - Ray integration



RAY FUNCTION

Maximum intensity projection

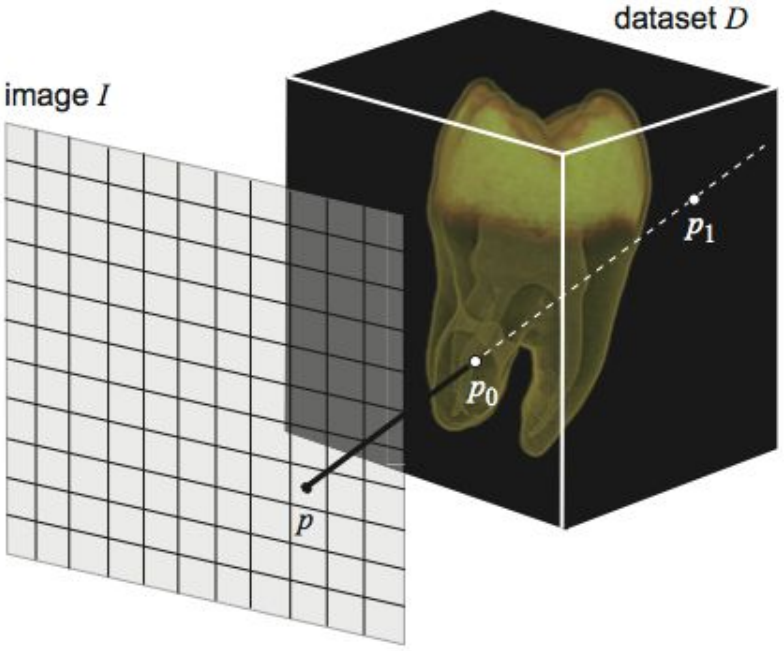
Average intensity projection

Distance to value function

Isosurface function

Ray function

- Consider a scalar signal $s : D \rightarrow \mathbf{R}$ to be drawn on the screen image I
- For each pixel $p \in I$
  - Construct a ray $\mathbf{r}$ orthogonal to I passing through p
  - Compute intersection points p_0 and p_1 of $\mathbf{r}$ with D
  - Express $I(p)$ as function of s along $\mathbf{r}$ between p_0 and p_1



Parameterize ray

$$q(t) = (1-t)p_0 + tp_1, \quad t \in [0,1]$$

Parameterize data on ray

$$s(t) = D(q(t)), \quad t \in [0,1]$$

Compute pixel intensity

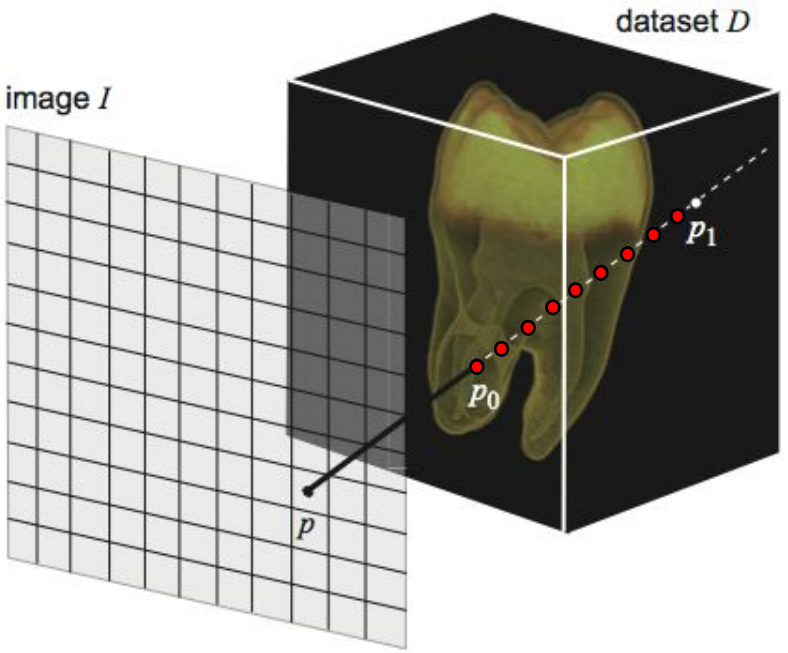
$$I(p) = F(s(t)), \quad t \in [0,1]$$

ray function

Ray function

Discretization:

1. Iterate through volume
2. Calculate ray function using the samples on the ray
3. Map calculated value onto color

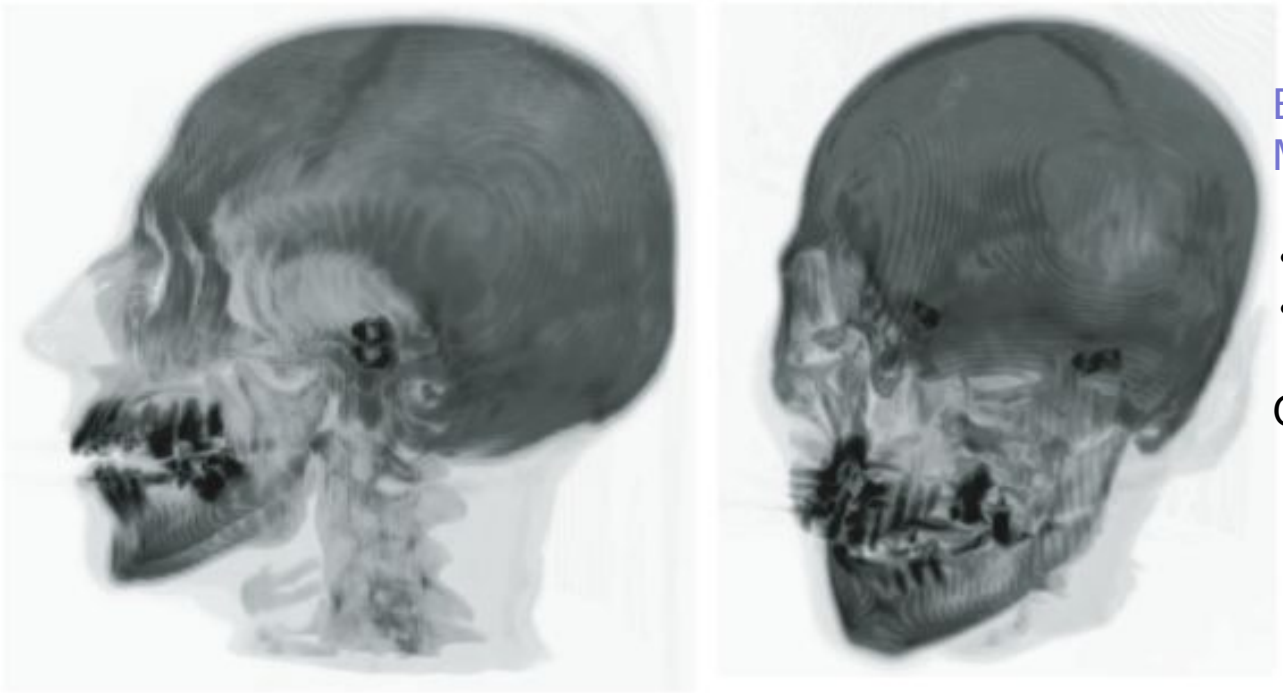


- \+ No geometry extraction
- \+ Greater flexibility than isosurfaces
  - \+ May be X-ray-like
  - \+ May be surface-like
  - \+ Results depend on the ray function

Maximum intensity projection (MIP)

Find maximum scalar along ray, then map it to color

$$I(p) = f(\max_{t \in [0,T]} s(t))$$



Example
MIP of human head CT

- white = low density (air)
- black = high density (bone)

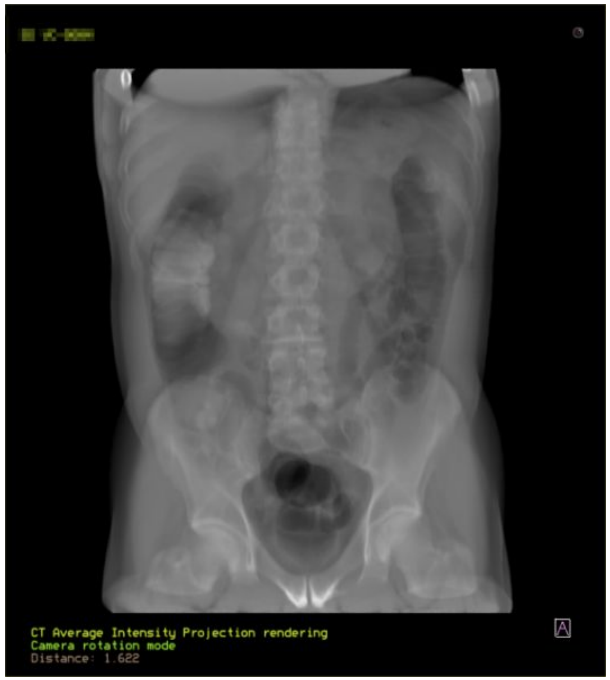
OK, but gives no depth cues

Average intensity projection (AIP)

- Compute average scalar along ray, then map it to color

$$I(p) = f\left(\frac{\int_{t=0}^{T} s(t)dt}{T}\right)$$

maximum intensity projection



average intensity projection

Example
Human torso

- black = low density (air)
- white = high density (bone)

Average intensity projection is equivalent to an X-ray

Distance to value

- Compute distance along ray while scalar value greater or equal than σ

$$I(p) = f\left(\min_{t \in [0,T], s(t) \geq \sigma} t\right)$$

distance to value 20

distance to value 50

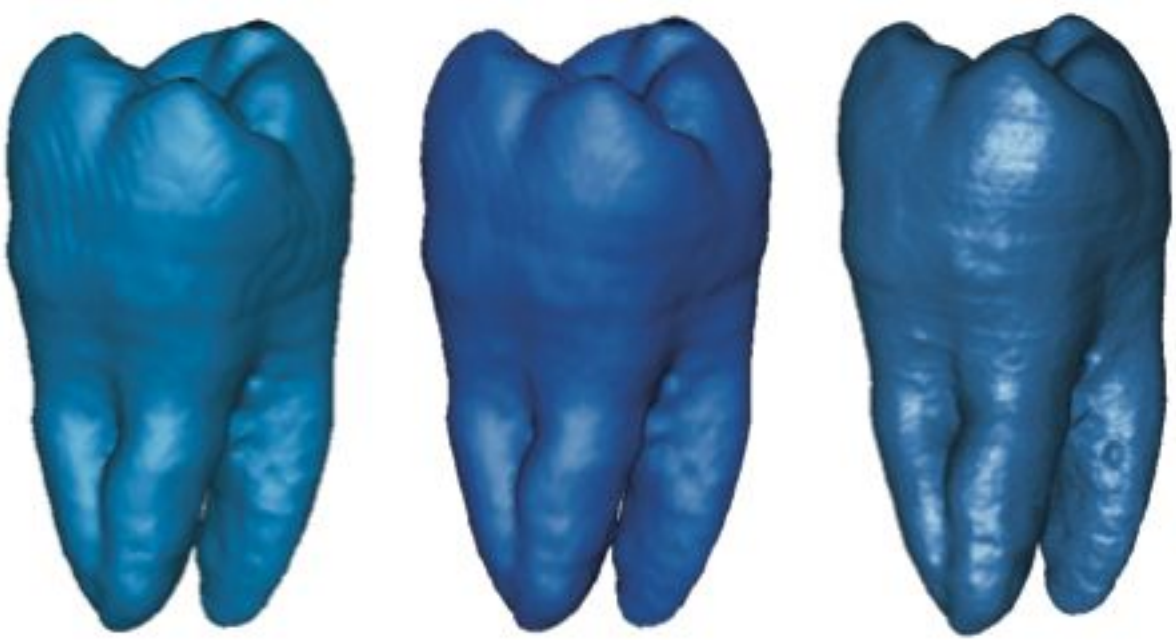
Example
Human head CT

- black = low distance
- white = high distance

Isosurface function (First hit)

- Compute whether a given isovalue σ exists along ray
- Produces similar result to marching cubes

$$I(p) = \begin{cases} f(\sigma), & \exists t \in [0,T], s(t) = \sigma \\ I_0, & \text{otherwise.} \end{cases}$$



isosurface (marching cubes)

isosurface (software ray casting)

isosurface (hardware ray casting)

RAY INTEGRATION

Transfer function

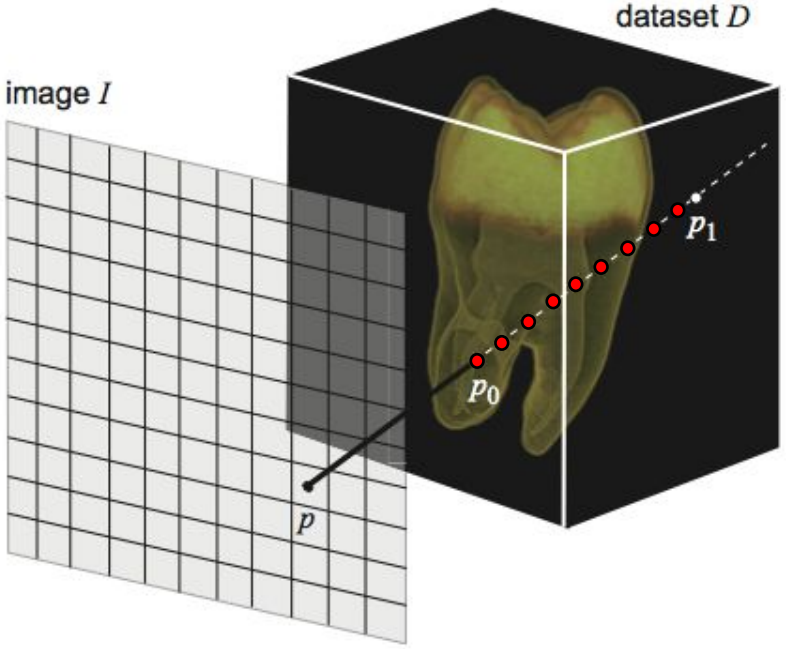
Volume rendering optical model

Volume rendering equation

Compositing schemes

Ray Integration

- We **integrate** through volume
- Every sample is mapped to color and opacity
  - With **transfer function**
- The colors with opacities are composed into one color
- No intermediate geometry extraction
- Greater flexibility than ray function
  - May be X-ray-like
  - May be surface-like
  - Results depend on the transfer function
  - Transfer function can be edited by the user
- Specification/editation of transfer function can be tricky

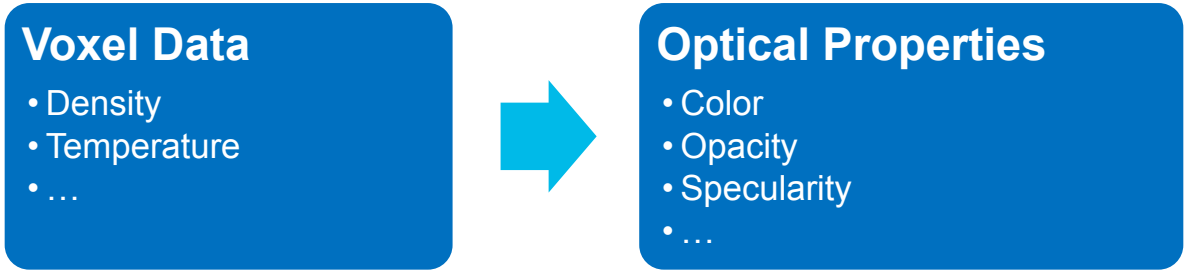


Transfer Function

- In scientific visualization, a volume can be parameterized by any number of scalars (pressure, temperature, density, etc.).
- These scalar attributes are mapped to visual channels (simple case is color and/or opacity) via **transfer function**.
- The values of scalar attributes often varies linearly, but the transfer function does not.

Transfer Function

- Maps voxel data values to optical properties

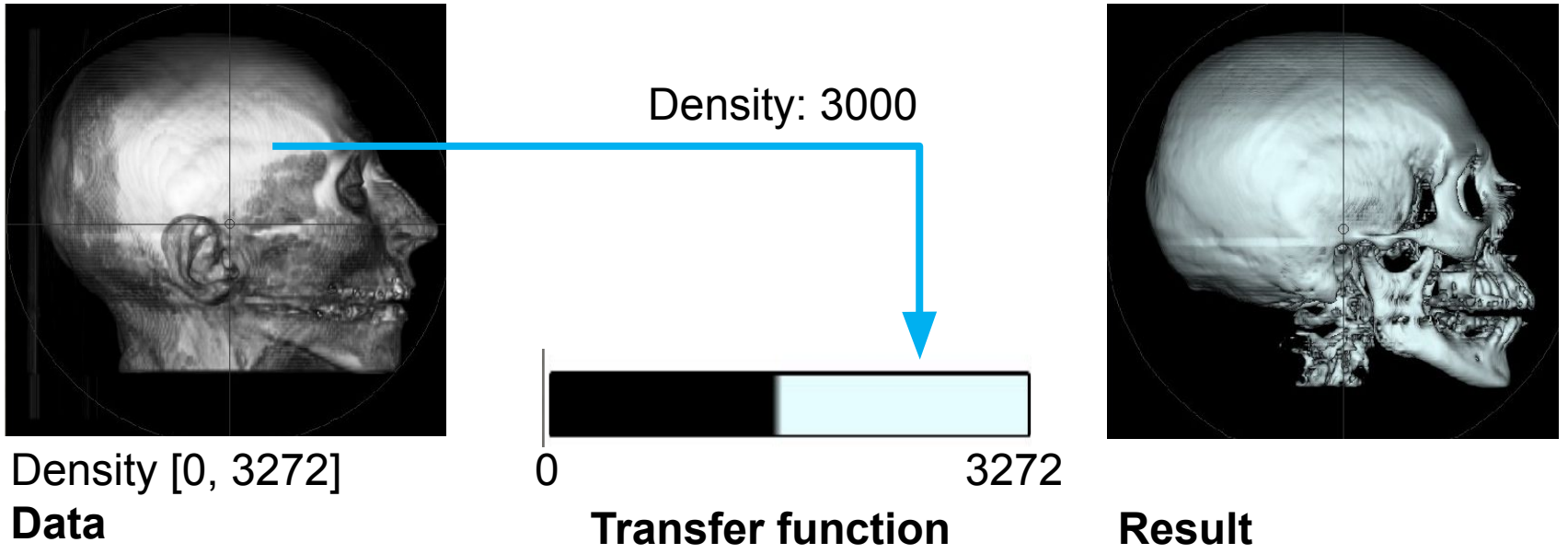


- Emphasize/supress or classify features of interest in the data
  - This color and transparency mapping is different from the color mapping discussed in previous lecture
- Piecewise linear function, Look-up table
- Dimension of transfer function depends on number of input attributes
  - 1D, 2D, 3D …

1D Transfer Function

Based on one attribute (density) assigns optical properties to data

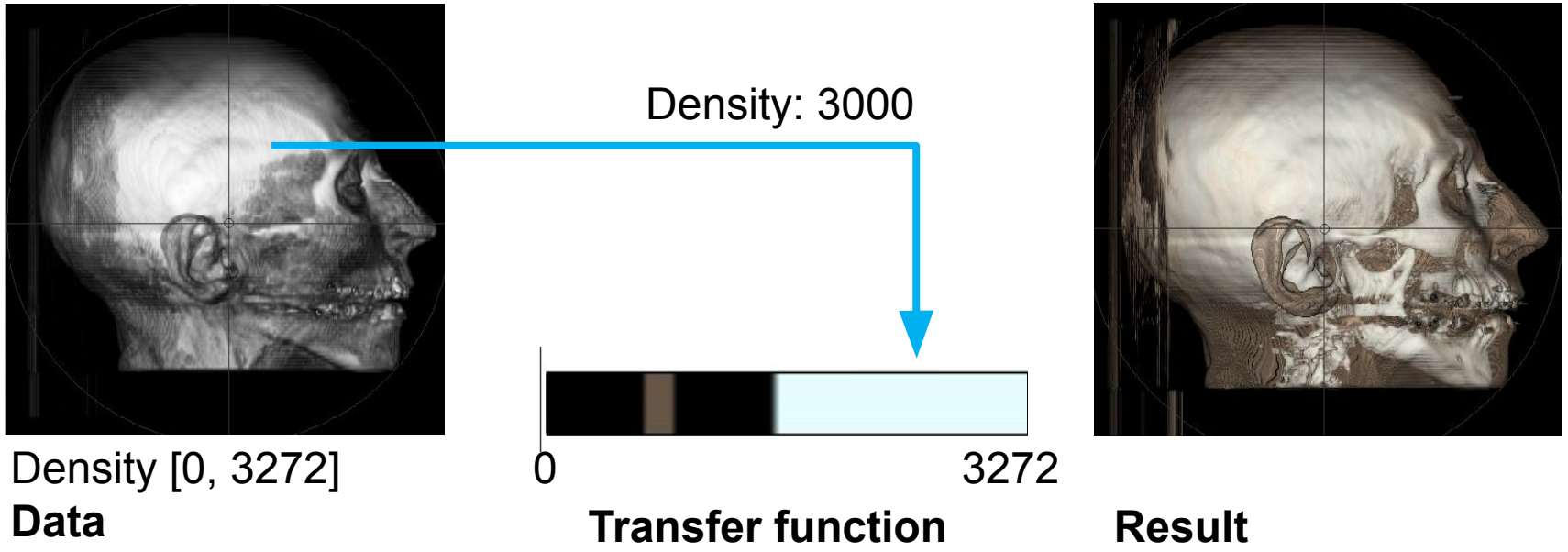
- Color
- Opacity



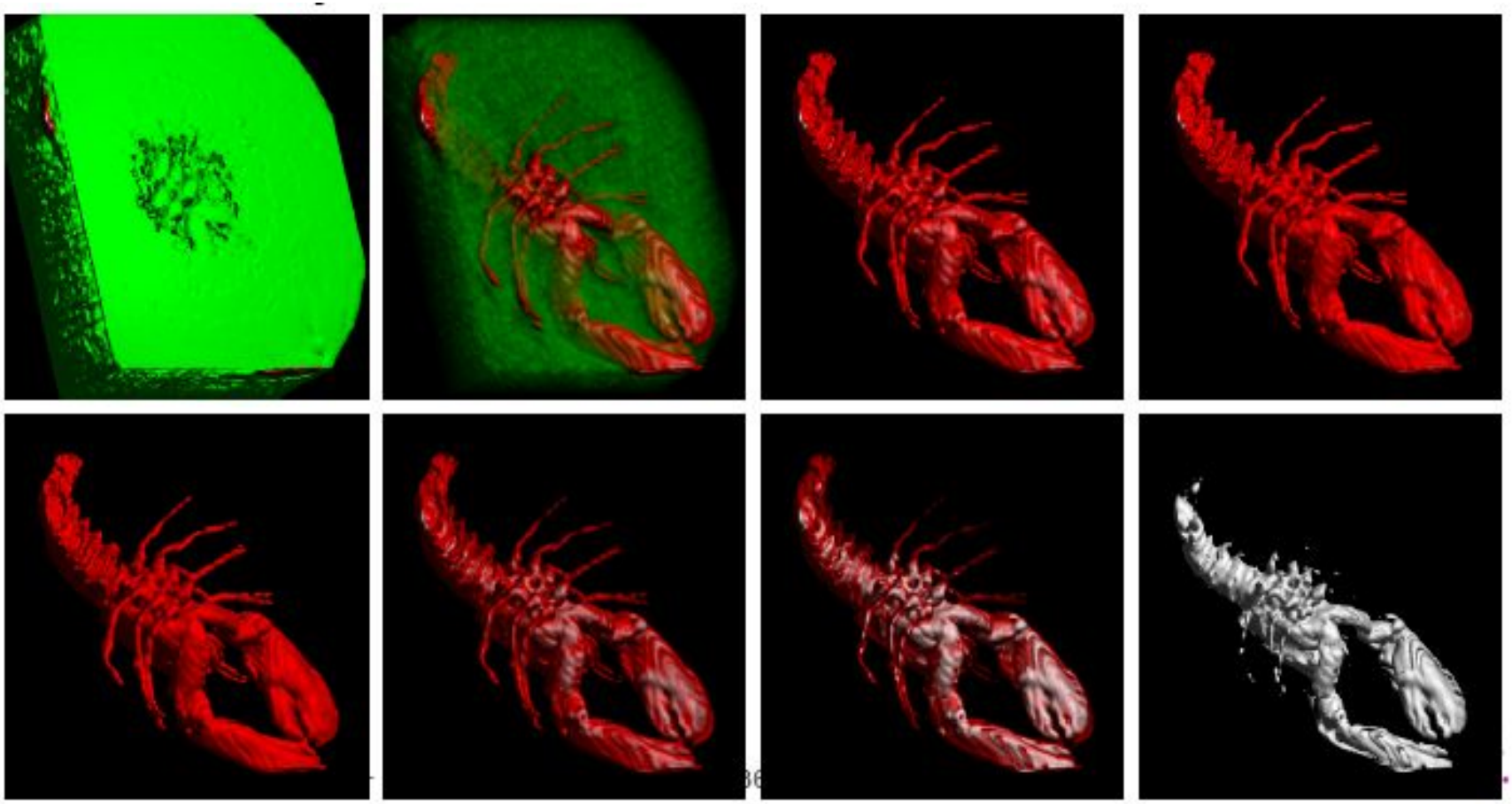
1D Transfer Function

Based on one attribute (density) assigns optical properties to data

- Color
- Opacity



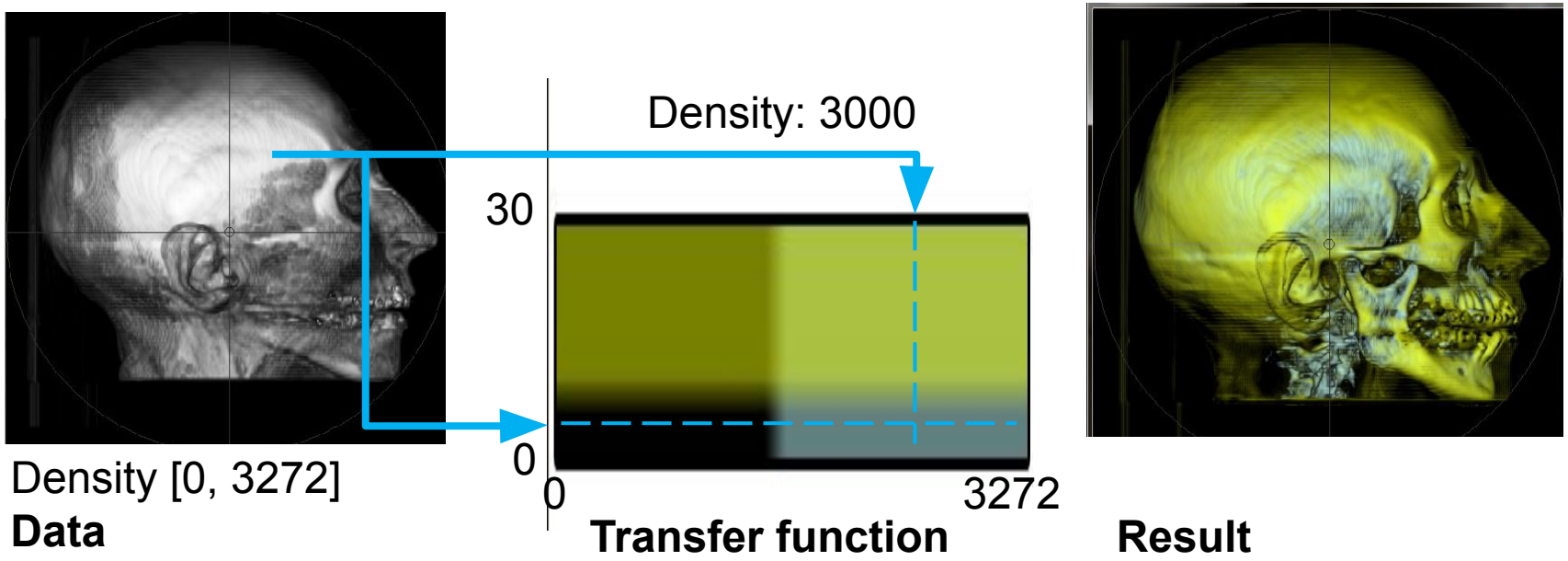
1D Transfer function examples: Lobster



2D Transfer Function

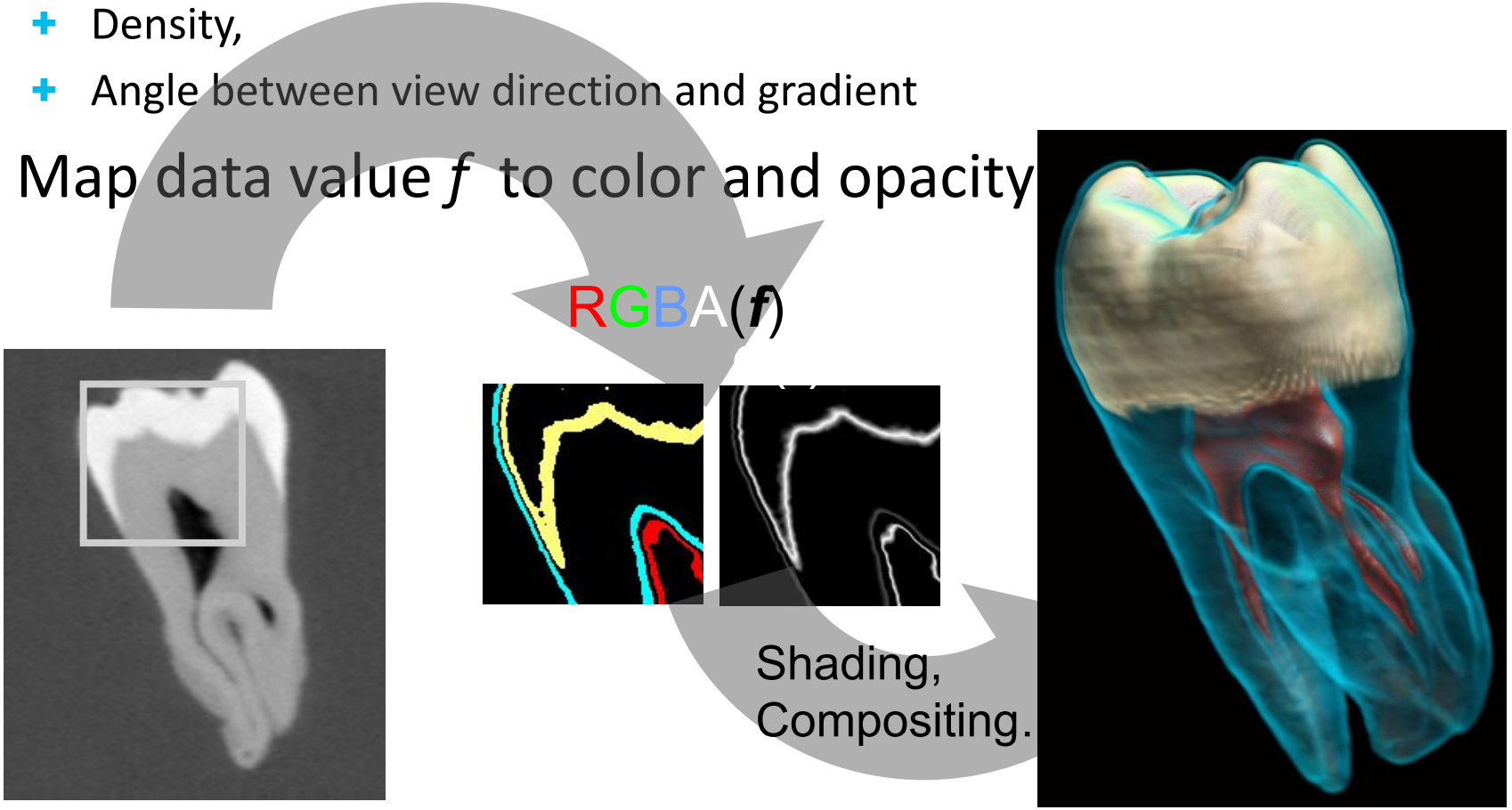
Based on two attributes (density, gradient size) assigns optical properties to data

- Color
- Opacity



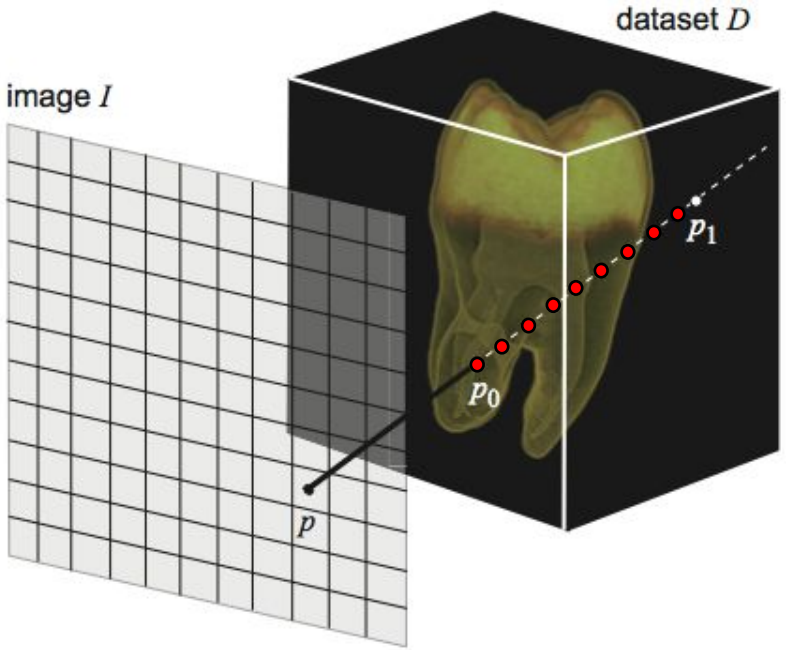
2D Transfer Function Example

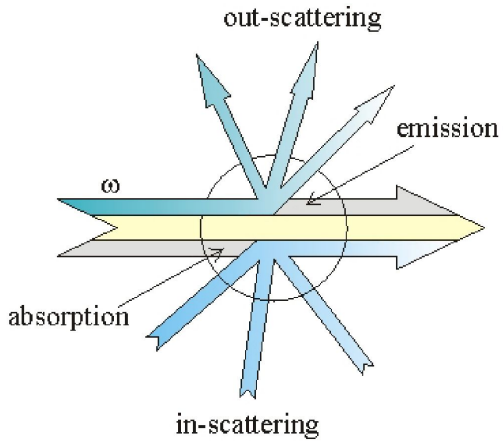
- Two attributes
  - Density,
  - Angle between view direction and gradient
- Map data value f to color and opacity



Ray Integration
Volume Rendering Optical Model

- Light interacts with volume ‘particles’ through:
  - **Absorption**
  - **Emission**
  - Scattering
- Sample volume along viewing rays
- Accumulate optical properties

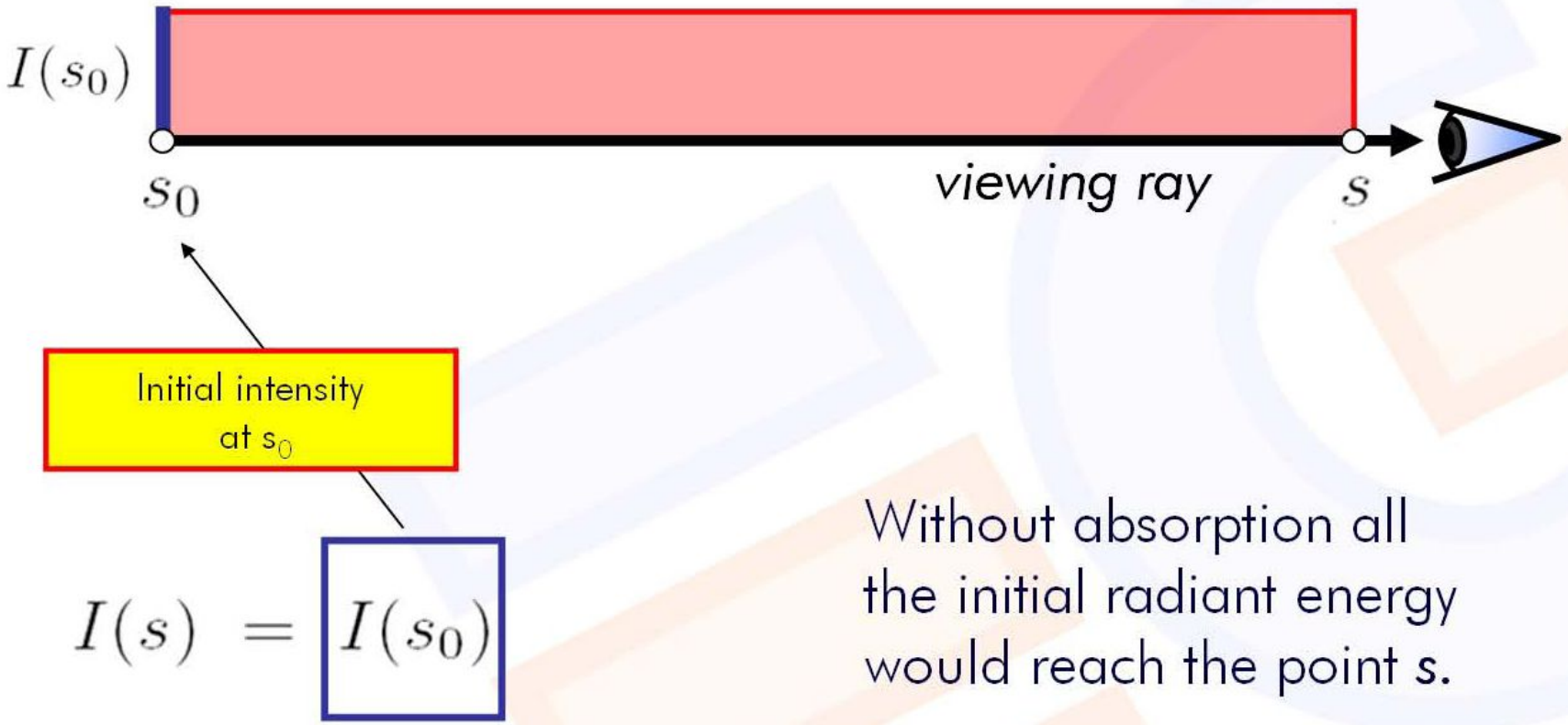




Ray Integration

How do we determine the radiant energy along the ray?

Physical model: emission and absorption, no scattering



Without absorption all the initial radiant energy would reach the point *s*.

Ray Integration

How do we determine the radiant energy along the ray?

Physical model: emission and absorption, no scattering

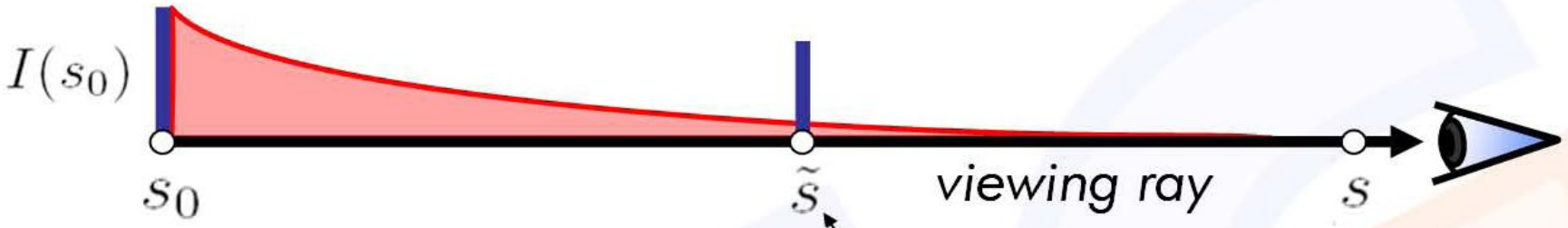
Absorption along the ray segment s_0 - s

$$I(s) = I(s_0)\, e^{-\tau(s_0,s)}$$

Ray Integration

How do we determine the radiant energy along the ray?

Physical model: emission and absorption, no scattering



One point $\tilde{s}$ along the viewing ray emits additional radiant energy.

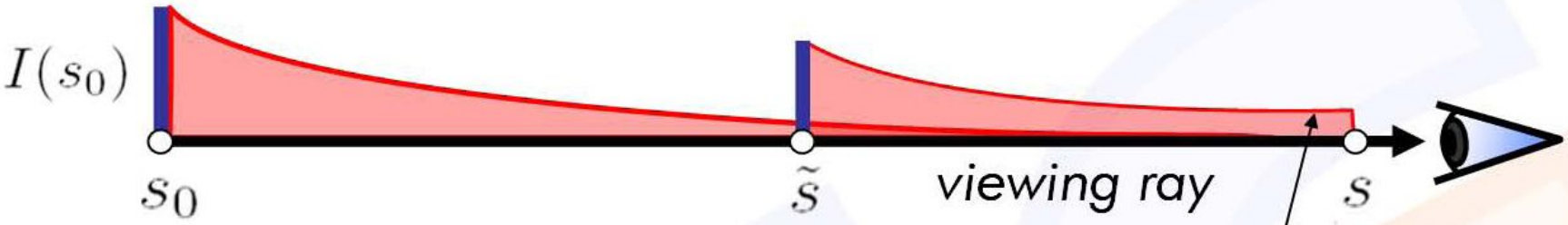
Active emission at point $\tilde{s}$

$$I(s) = I(s_0)\, e^{-\tau(s_0,s)} + q(\tilde{s})$$

Ray Integration

How do we determine the radiant energy along the ray?

Physical model: emission and absorption, no scattering



One point $\tilde{s}$ along the viewing ray emits additional radiant energy.

Absorption along the distance $s - \tilde{s}$

$$I(s) = I(s_0)\, e^{-\tau(s_0,s)} + q(\tilde{s})\, e^{-\tau(\tilde{s},s)}$$

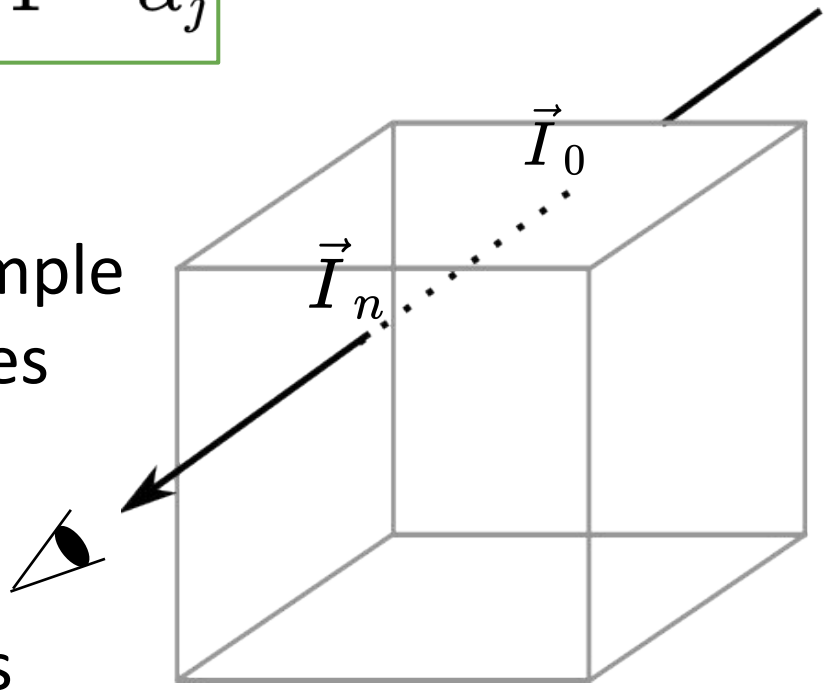
HOW DOES IT WORK IN VOXELS

Discretization

Volume rendering equation

$$\vec{I}(x,y) = \sum_{i=0}^{n} \vec{I}_i \cdot \prod_{j=i+1}^{n} 1 - \alpha_j$$

$$\vec{I}_i = \vec{C}_i \cdot \alpha_i$$



- Intensity emitted by a sample is absorbed by the samples closer to the camera

- The intensity that reaches the camera is sum of intensities of all samples along the ray that reaches the camera

COMPOSITING SAMPLES ALONG RAY

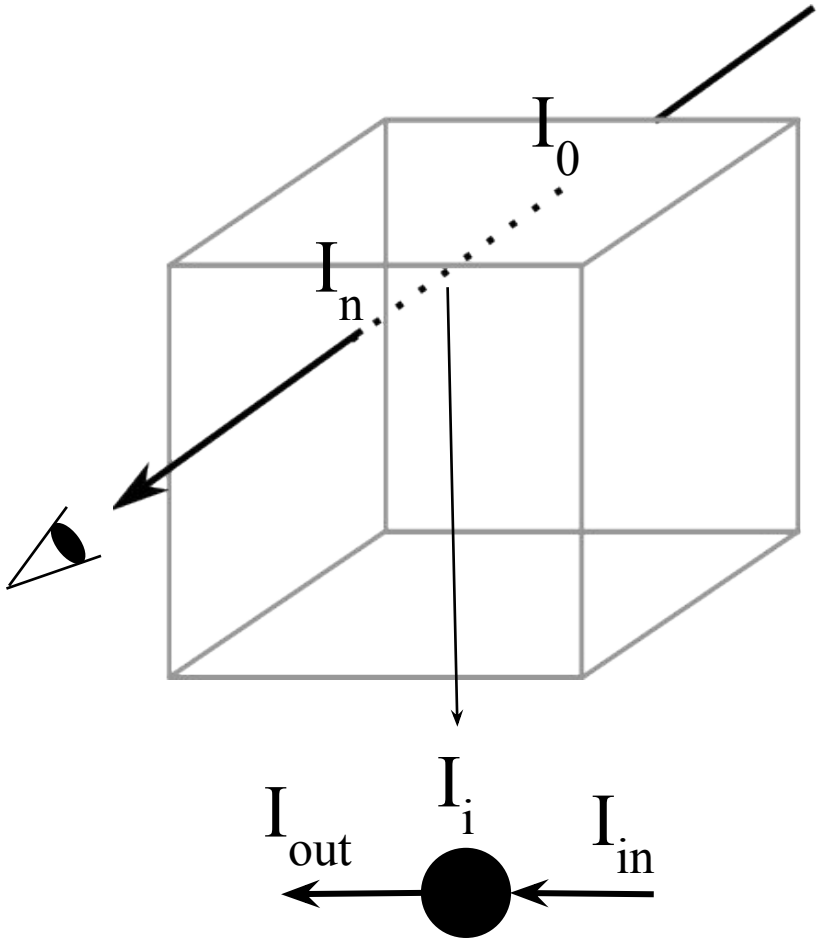
Rendering equation

$$I(x,y) = \sum_{i=0}^{n} I_i \prod_{j=i+1}^{n} (1-\alpha_j)$$

$$I_i = C_i \cdot \alpha_i$$

$$I_{out} = I_{in}(1-\alpha_i) + I_i$$

$$C_{out} = C_{in}(1-\alpha_i) + C_i\alpha_i$$



Compositing samples along the ray

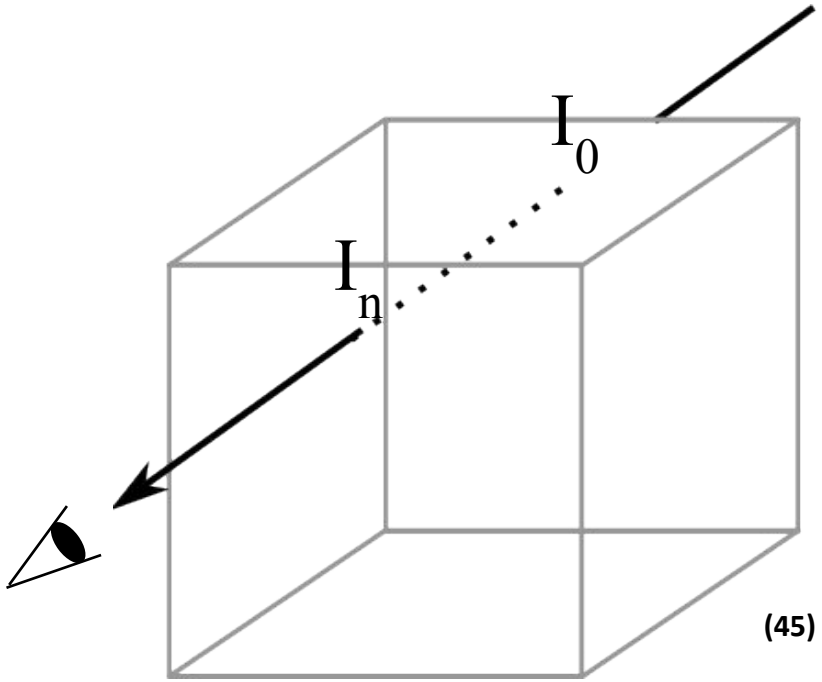
$$\vec{I}(x,y) = \sum_{i=0}^{n} \vec{I}_i \cdot \prod_{j=i+1}^{n} 1 - \alpha_j$$

Front to back composition

$$\vec{I}(x,y) = \vec{I}_n + (1-\alpha_n) \cdot \vec{I}_{n-1} + \cdots + (1-\alpha_n) \cdot \cdots \cdot (1-\alpha_1) \cdot \vec{I}_0$$

Back to front composition

$$\vec{I}(x,y) = \vec{I}_n + (1-\alpha_n) \cdot (\vec{I}_{n-1} + (1-\alpha_{n-1}) \cdot \ldots (\vec{I}_1 + (1-\alpha_1) \cdot I_0) \ldots)$$



Back to front composition

$$\vec{I}(x,y) = \sum_{i=0}^{n} \vec{I}_i \cdot \prod_{j=i+1}^{n} 1 - \alpha_j$$

$$\vec{I}(x,y) = \vec{I}_n + (1-\alpha_n) \cdot \vec{I}_{n-1} + \cdots + (1-\alpha_n) \cdot \cdots \cdot (1-\alpha_1) \cdot \vec{I}_0$$

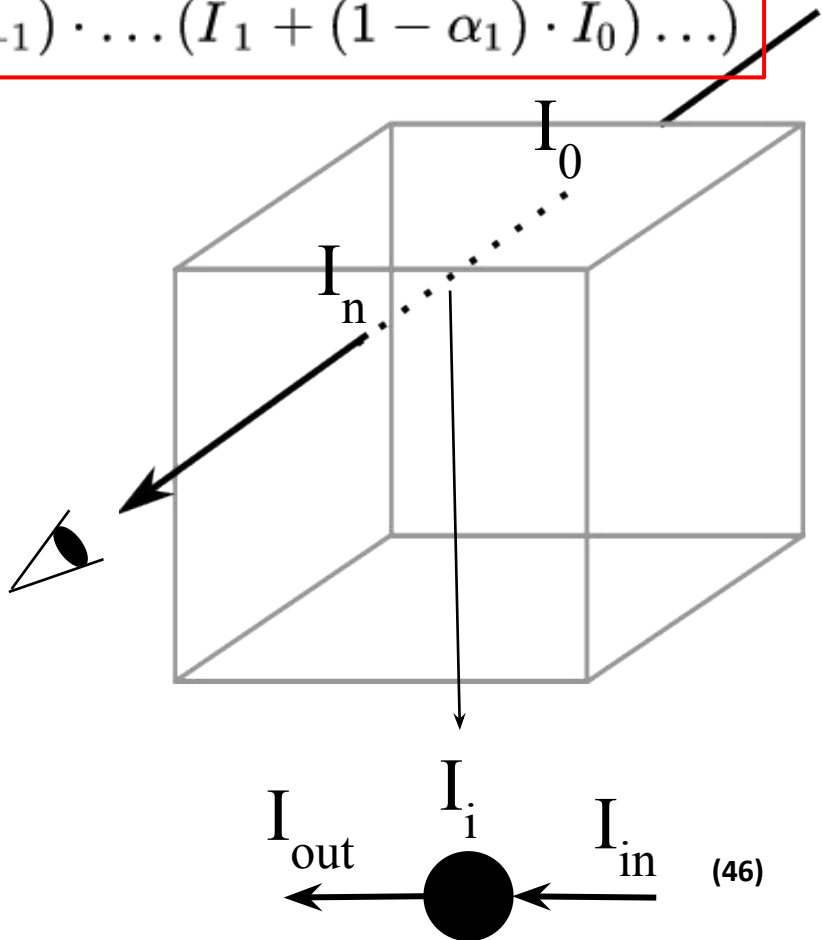
$$\vec{I}(x,y) = \vec{I}_n + (1-\alpha_n) \cdot (\vec{I}_{n-1} + (1-\alpha_{n-1}) \cdot \ldots (\vec{I}_1 + (1-\alpha_1) \cdot I_0) \ldots)$$

$$\vec{I}_{out} = \vec{I}_i + (1-\alpha_i) \cdot \vec{I}_{in}$$

$$\vec{I}_i = \vec{C}_i \cdot \alpha_i$$

$$\vec{C}_{out} = \vec{C}_i \cdot \alpha_i + (1-\alpha_i) \cdot \vec{C}_{in}$$

Initially $\vec{I}_{in}$ is 0 and i is 0



Front to back composition

$$\vec{I}(x,y) = \sum_{i=0}^{n} \vec{I}_i \cdot \prod_{j=i+1}^{n} 1 - \alpha_j$$

$$\vec{I}(x,y) = \vec{I}_n + (1-\alpha_n) \cdot \vec{I}_{n-1} + \cdots + (1-\alpha_n) \cdot \cdots \cdot (1-\alpha_1) \cdot \vec{I}_0$$

$$\vec{I}(x,y) = \vec{I}_n + (1-\alpha_n) \cdot (\vec{I}_{n-1} + (1-\alpha_{n-1}) \cdot \ldots (\vec{I}_1 + (1-\alpha_1) \cdot I_0) \ldots)$$

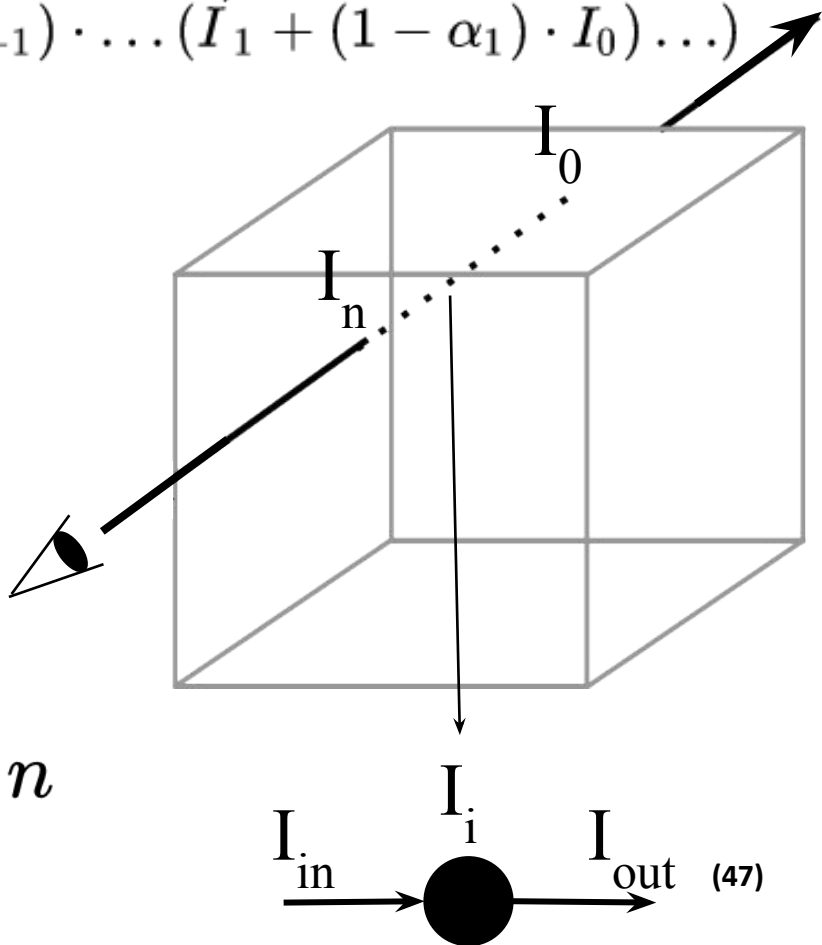
$$\vec{I}_{out} = \vec{I}_{in} + T_{in} \cdot \vec{I}_i$$

$$T_{out} = T_{in} \cdot (1-\alpha_i)$$

$$\vec{I}_i = \vec{C}_i \cdot \alpha_i$$

$$\vec{C}_{out} = \vec{C}_{in} + T_{in} \cdot \vec{C}_i$$

Initially $\vec{I}_{in}$ is 0, T_{in} is 1 and i is n



Front to back composition

$$\vec{I}(x,y) = \sum_{i=0}^{n} \vec{I}_i \cdot \prod_{j=i+1}^{n} 1 - \alpha_j$$

$$\vec{I}(x,y) = \vec{I}_n + (1-\alpha_n)\cdot\vec{I}_{n-1} + \cdots + (1-\alpha_n)\cdot\cdots\cdot(1-\alpha_1)\cdot\vec{I}_0$$

$$\vec{I}(x,y) = \vec{I}_n + (1-\alpha_n)\cdot(\vec{I}_{n-1} + (1-\alpha_{n-1})\cdot\ldots(\vec{I}_1 + (1-\alpha_1)\cdot I_0)\ldots)$$

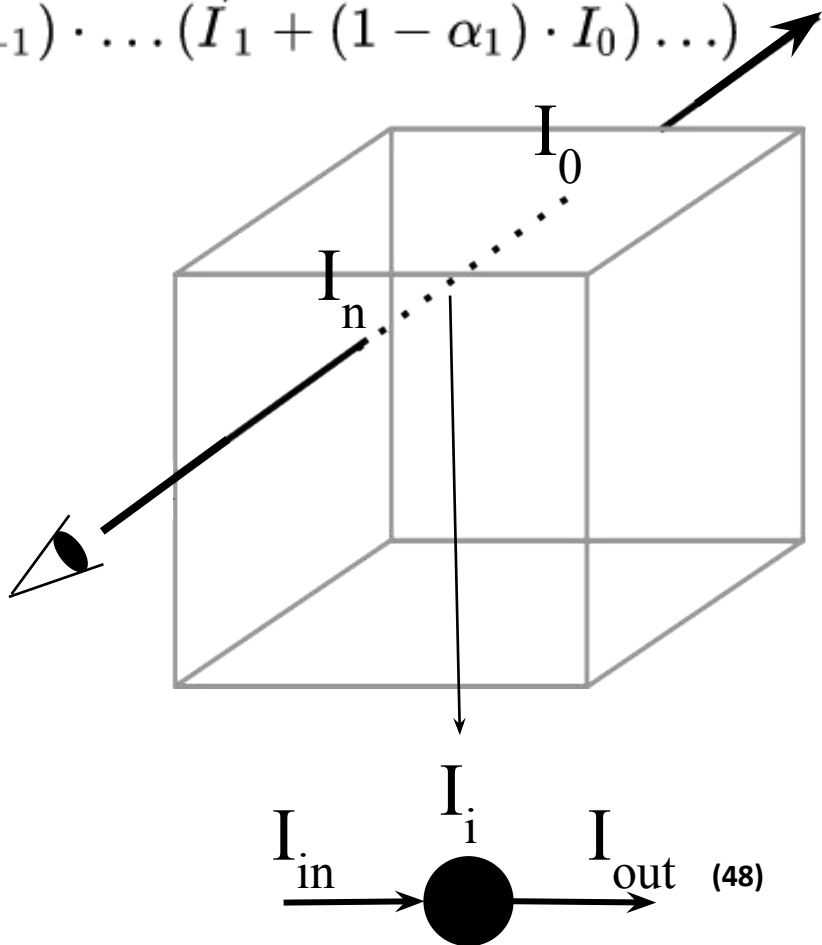
$$\vec{I}_{out} = \vec{I}_{in} + (1-\alpha_{in})\cdot\vec{I}_i$$

$$\alpha_{out} = \alpha_{in} + (1-\alpha_{in})\cdot\alpha_i)$$

$$\vec{I}_i = \vec{C}_i \cdot \alpha_i$$

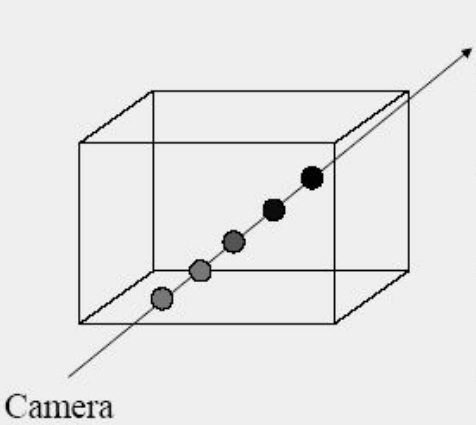
$$\vec{C}_{out} = \vec{C}_{in} + (1-\alpha_{in})\cdot\vec{C}_i$$

Initially $\vec{I}_{in}$ is 0, α_{in} is 0 and i is n



Volume rendering

- Back to front composition
  - We need to calculate only one equation in each sample
- Front to back – cumulative opacity
  - We need to calculate two equations for each sample
  - When the opacity reaches a defined “large” value the process ends we terminate the composition

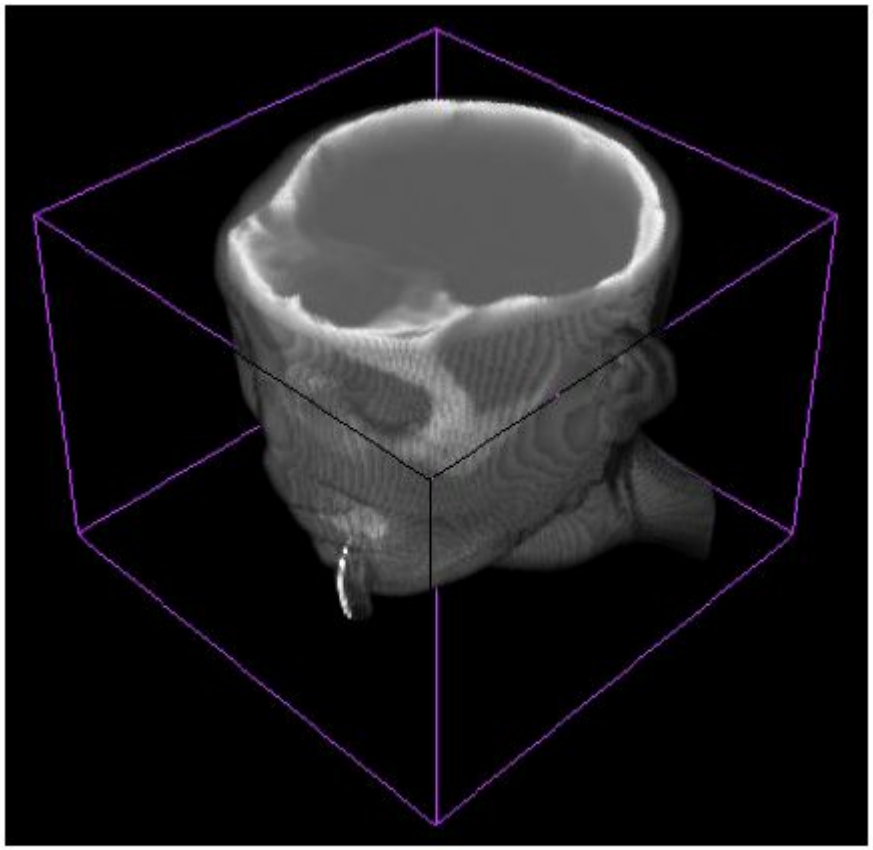


Current Alpha	Accumulated Alpha
0.2	0.2
0.4	0.4+(1- 0.4)*0.2=0.52
0.5	0.5+(1- 0.5)*0.52=0.76
0.4	0.4+(1- 0.4)*0.76=0.86
0.6	0.6+(1- 0.6)*0.86=0.94

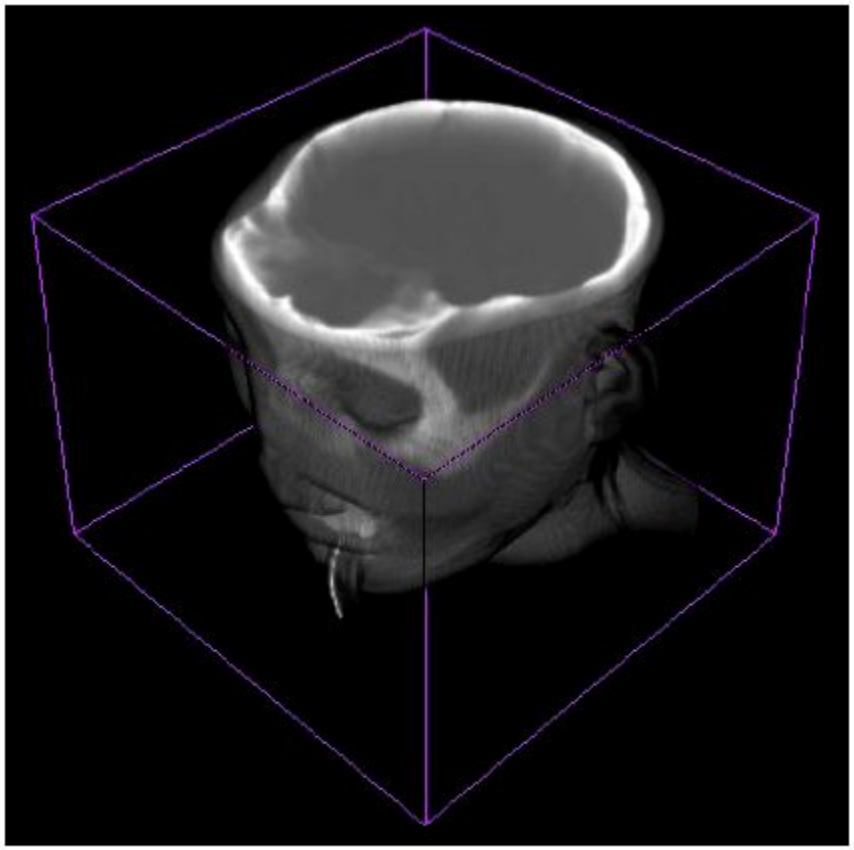
- Stop when opacity (alpha) reaches ~1.0 as nothing behind is visible

QUALITY OF VOLUME VISUALIZATION

Interpolation



Nearest neighbour
(visible artefacts in rendering)



Tri-linear interpolation (i.e. in X,Y & Z)
(smoother, no artefacts)

Shading

- Problem of shading the “volumetric information”
- In CG there are several approaches to shading (e.g. Gouraud shading, Phong shading …)
- Based on light reflection
- In visualization the data set is very large => high computational costs
- Reason for shading?
- Better impression about the internal structure

Shading

MIP technique

Shaded embedded iso-surface.

Normal vectors of voxel data

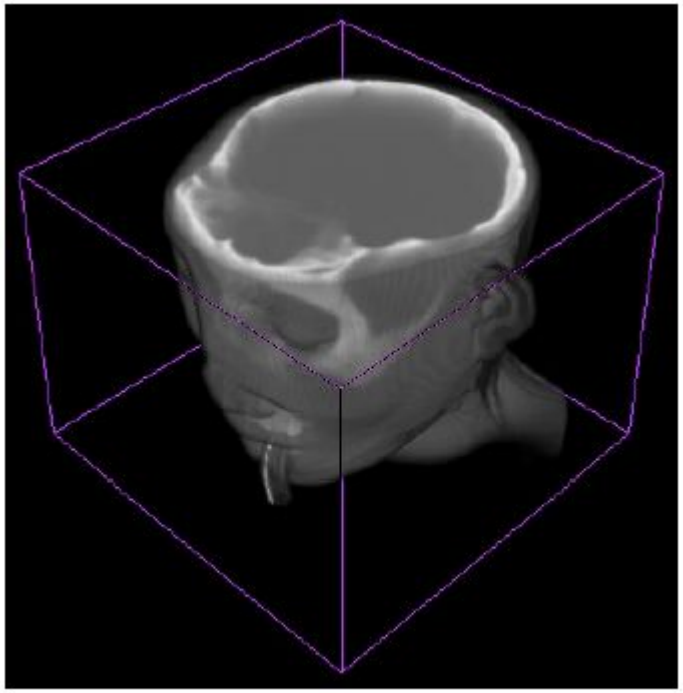
- Normals of surfaces embeded in volumetric data can be calculated as gradient of the data.

$$D_x = f(x-1, y, z) - f(x+1, y, z)$$
$$D_y = f(x, y-1, z) - f(x, y+1, z)$$
$$D_z = f(x, y, z-1) - f(x, y, z+1)$$
$$D = [D_x, D_y, D_z]$$

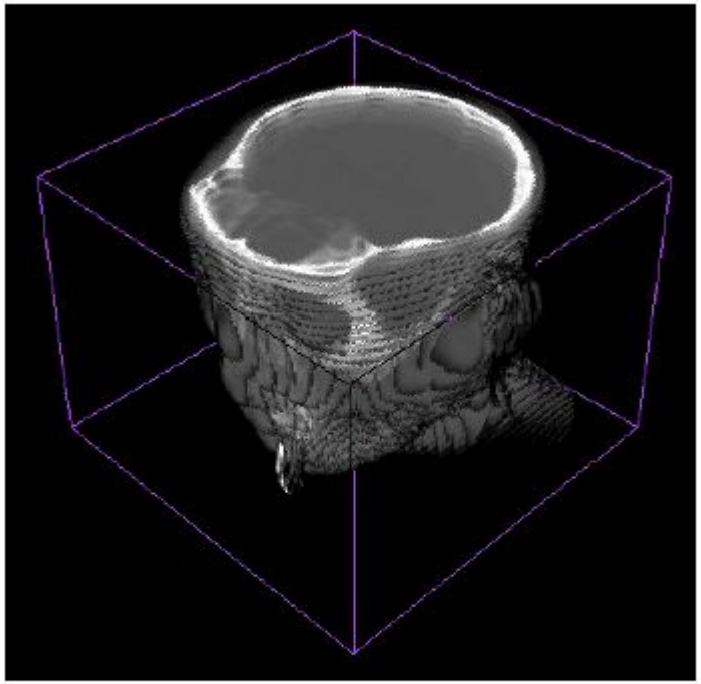
- To obtain normal vector at position of sample on the ray we have to interpolate the gradients (tri-linear interpoltion) and then normalize the interpolated gradient.
- Then we can shade the sample e.g. with Phong shading

Step size

- Effect of step size (distance between samples on the ray) in volume rendering



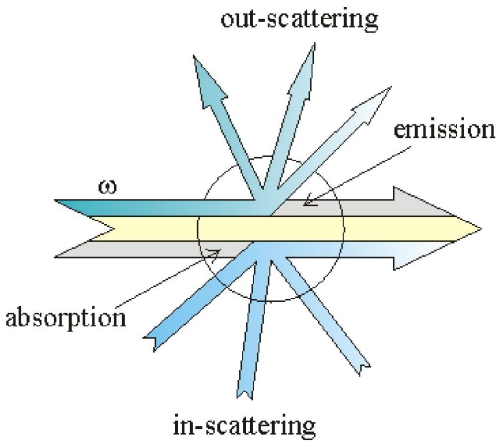
Step = 0.2



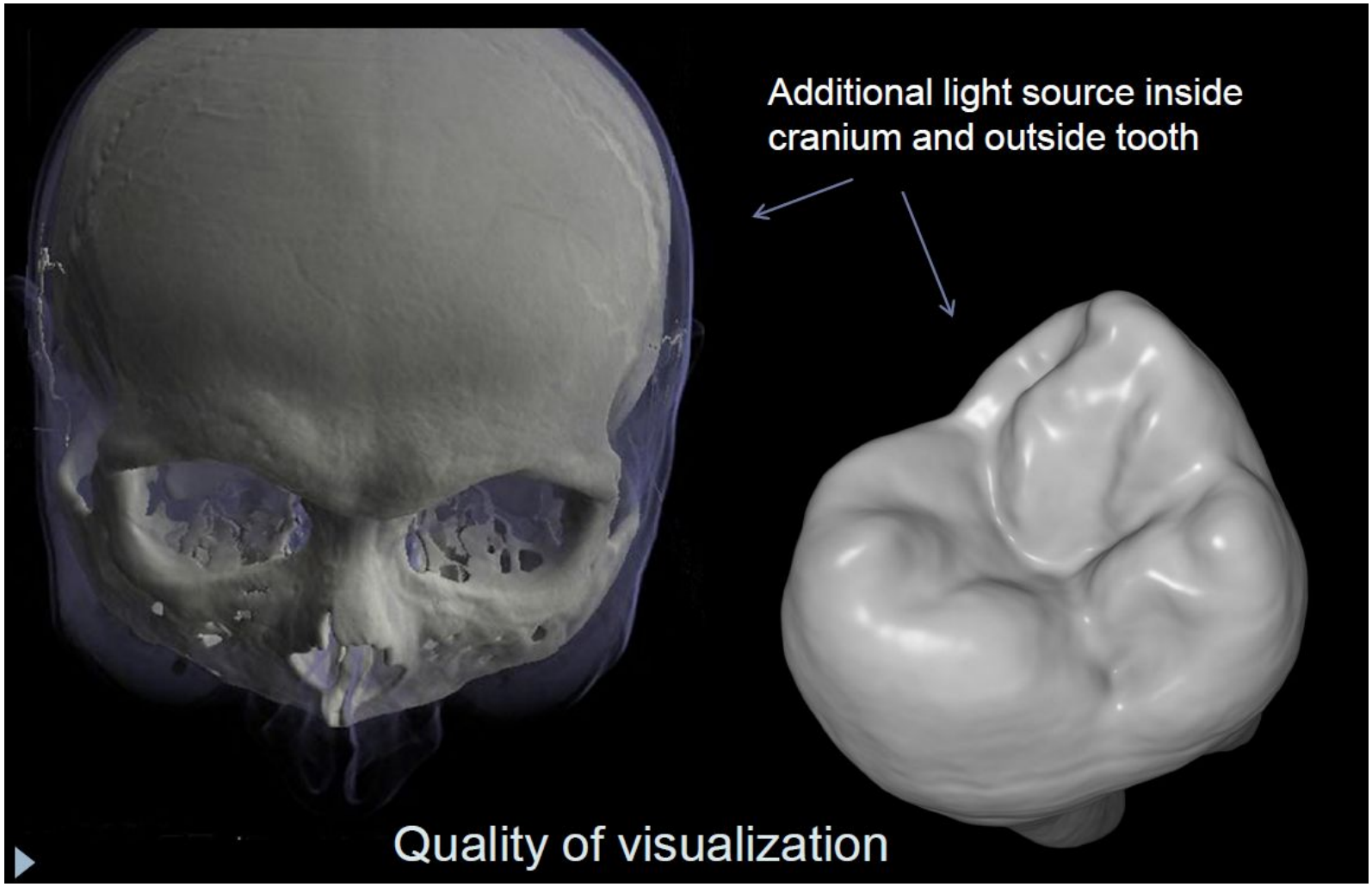
Step = 7.0
(highly visible artefacts
- distract from data itself)

Light Scattering

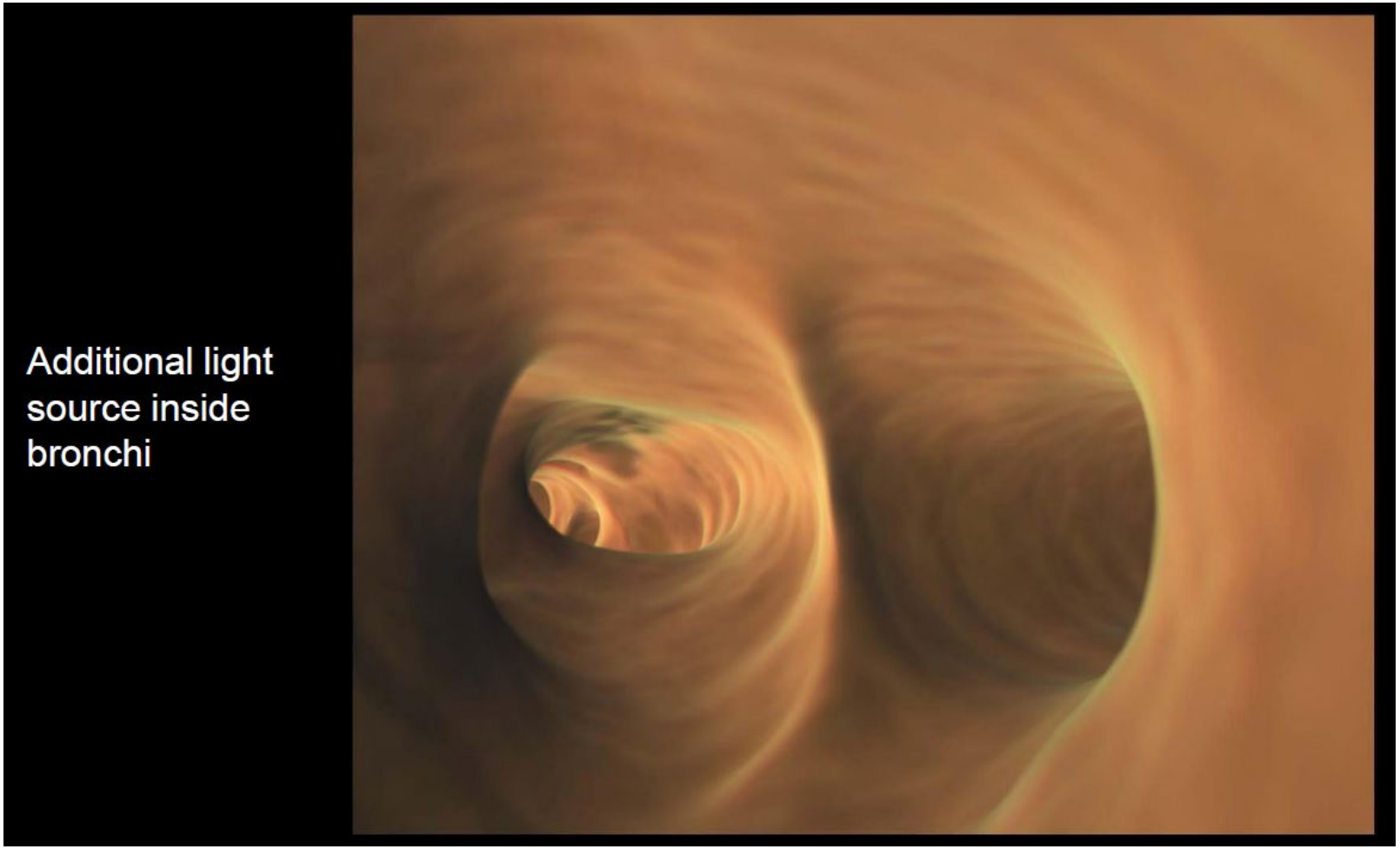
- Single scattering and shadowing
  - Single scattering of light that comes from an external light source (not from within the volume)
  - Shadows are modeled by taking into account the attenuation of light that is incident from an external light source
- Multiple scattering
  - Here the goal is to evaluate the complete illumination model for volumes, including emission, absorption and scattering.



Single scattering and shadowing

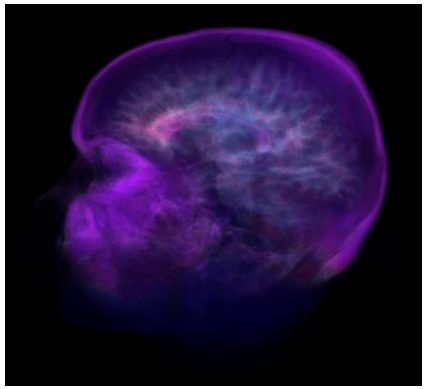


Single scattering and shadowing



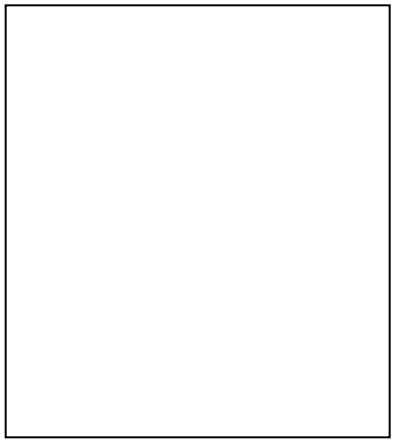
DIRECT VOLUME RENDERING ALGORITHMS

Texture Mapping

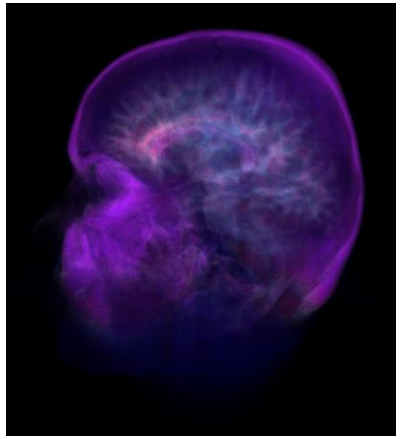


2D image

+



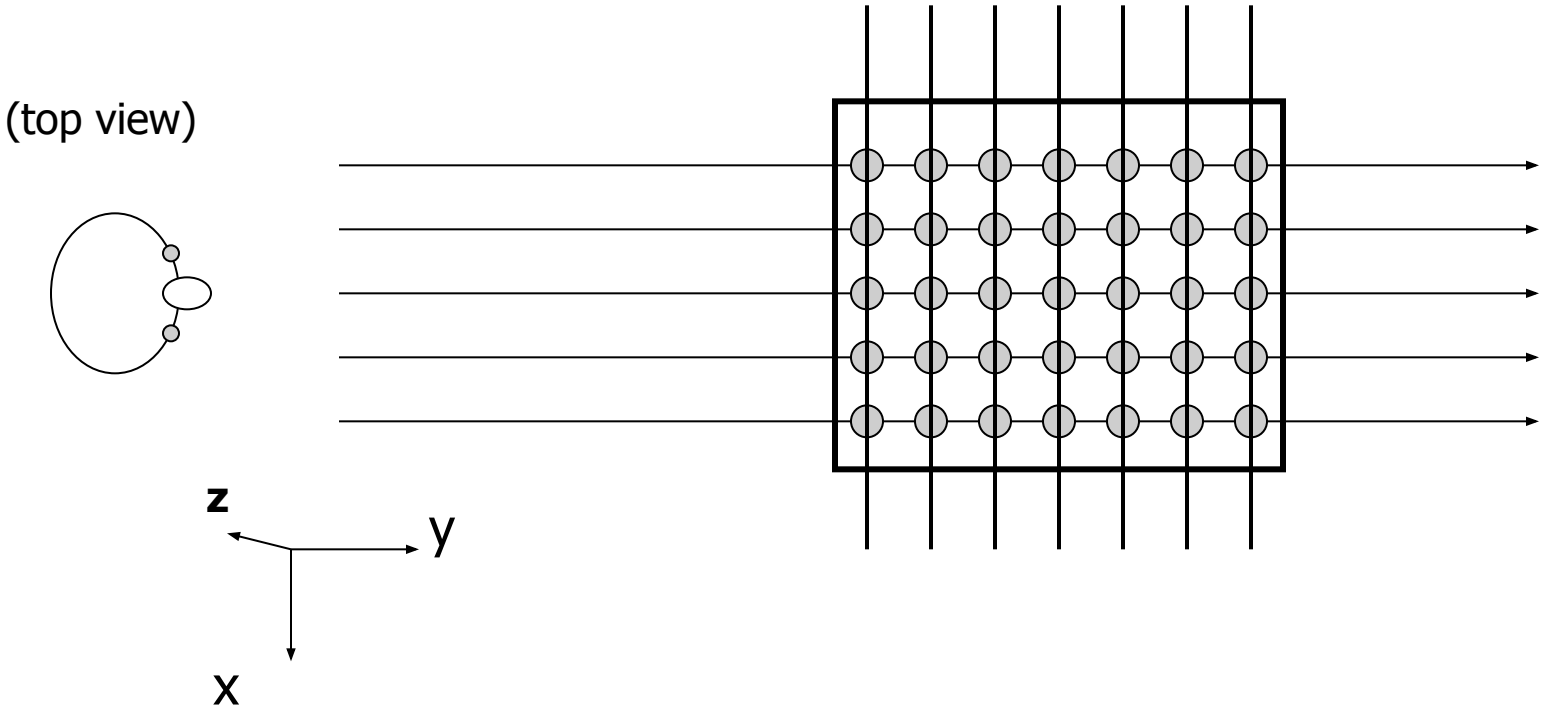
2D polygon



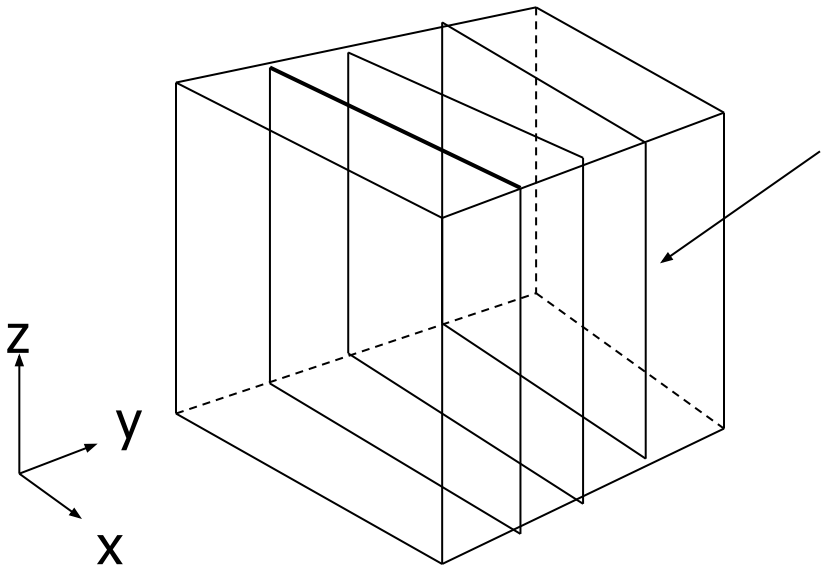
Textured-mapped polygon

Texture Mapping for Volume Rendering

Consider ray casting ...



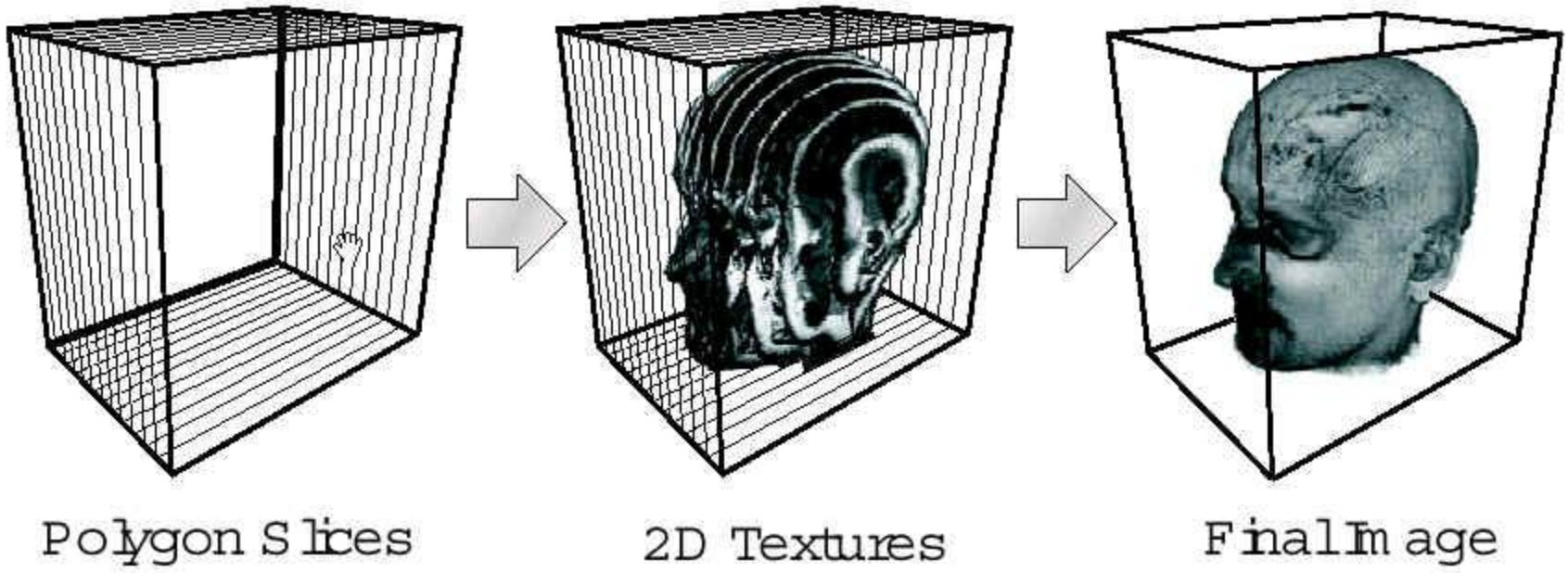
Texture based volume rendering



Use proxy geometry for sampling

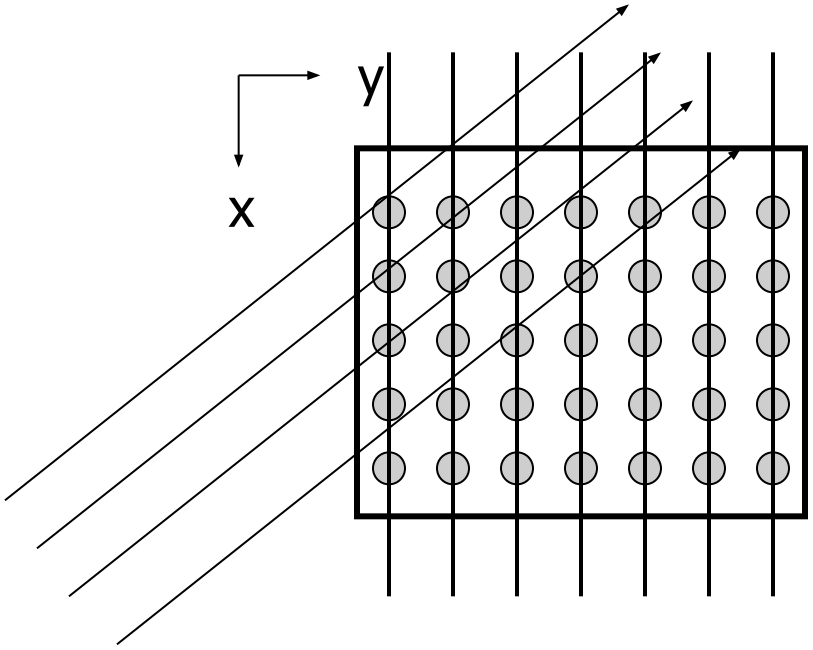
- Render every xz slice in the volume as a texture-mapped polygon
- The proxy polygon will sample the volume data
- Per-fragment RGBA (color and opacity) from transfer function
- The polygons are blended from back-to-front

Texture based volume rendering



Changing Viewing Direction

What if we change the viewing position?

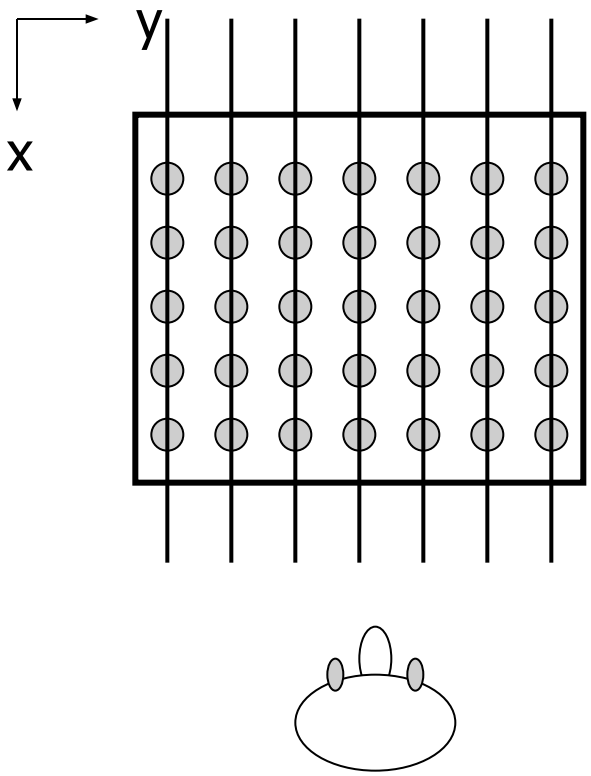


That is okay, we just change the eye position (or rotate the polygons and re-render),

Until …

Changing View Direction (2)

Until …

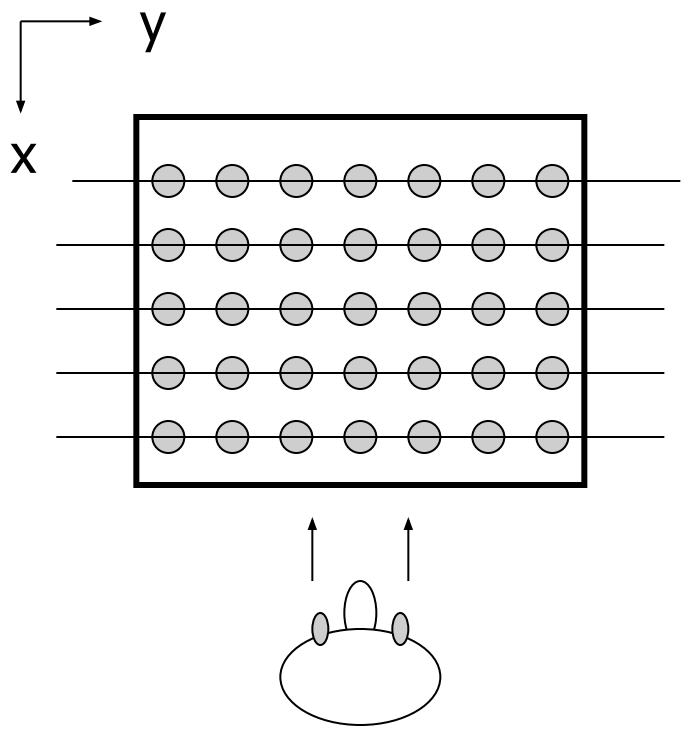


You are not going to see anything this way …

This is because the view direction now is Parallel to the slice planes

What do we do?

Switch Slicing Planes

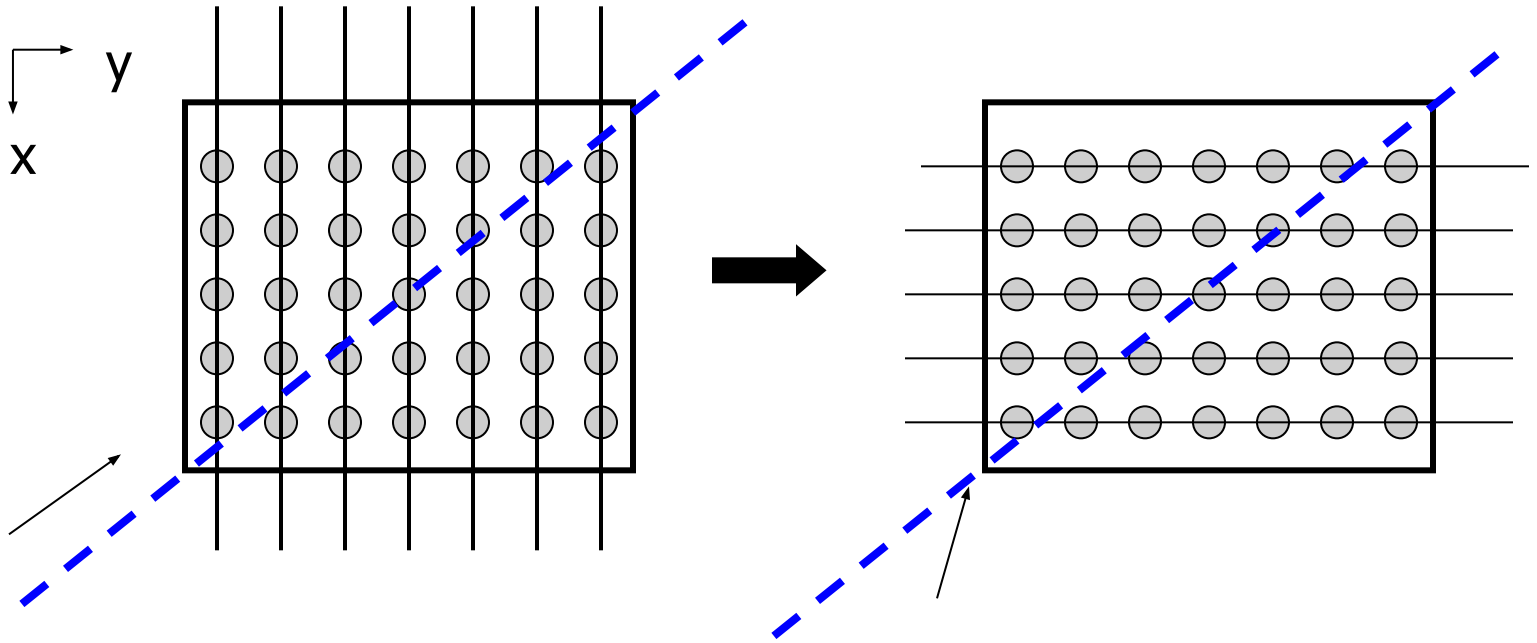


What do we do?

- Change the orientation of slicing planes
- Now the slice polygons are parallel to YZ plane in the object space

Some Considerations…

When do we need to change the slicing orientation?



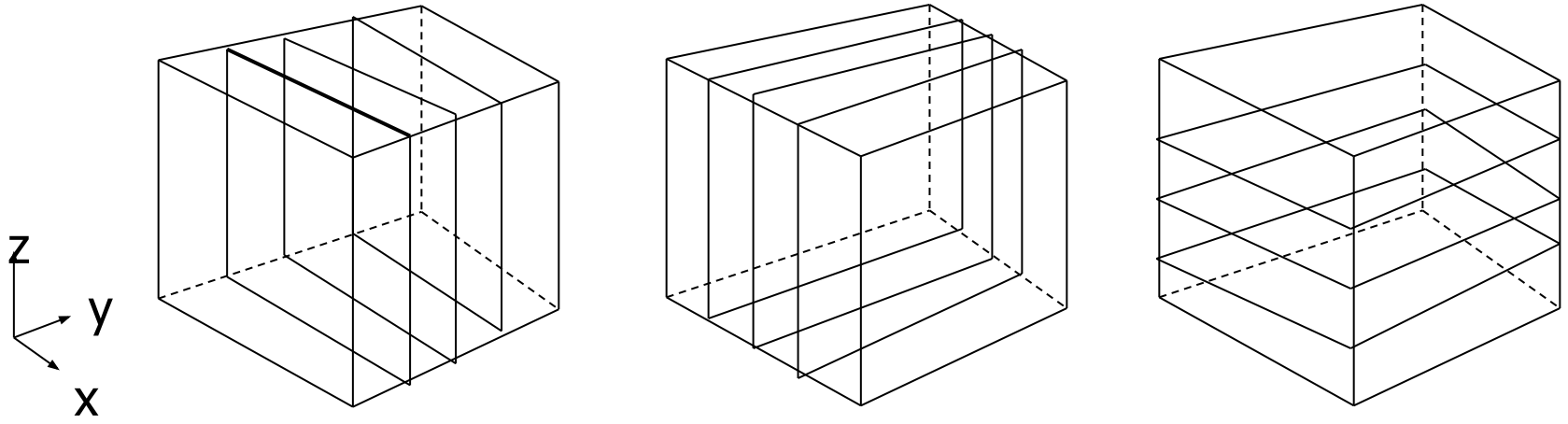
When the major component of view vector changes from y to -x

Some Considerations…

- Major component of view vector?
- Given the view vector (x,y,z) -> get the maximal component
- If x: then the slicing planes are parallel to yz plane
- If y: then the slicing planes are parallel to xz plane
- If z: then the slicing planes are parallel to xy plane
- This is called (object-space) axis-aligned method.

Three copies of data needed

- We need to reorganize the input textures for different view directions.
- Reorganize the textures on the fly is too time consuming. We want to prepare the texture sets beforehand

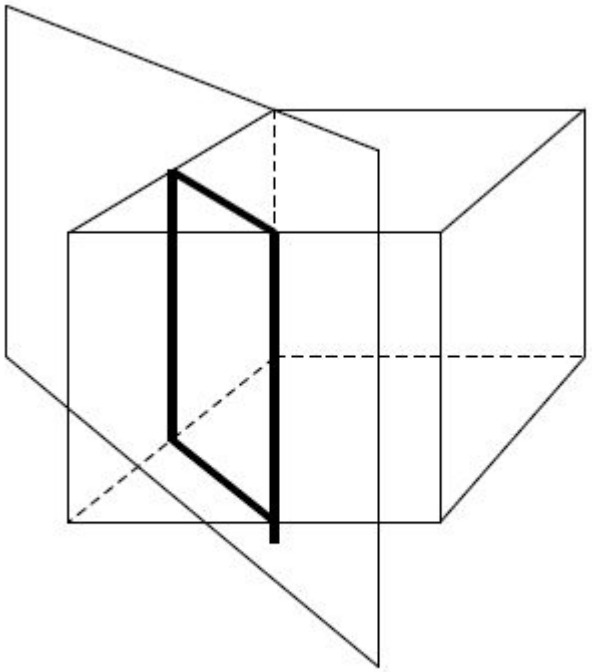


xz slices yz slices xy slices

3D Texture Mapping

- Arbitrary slicing through the volume and texture mapping capabilities are needed
  - Arbitrary slicing polygon: this can be computed in real time

This is basically polygon-volume clipping



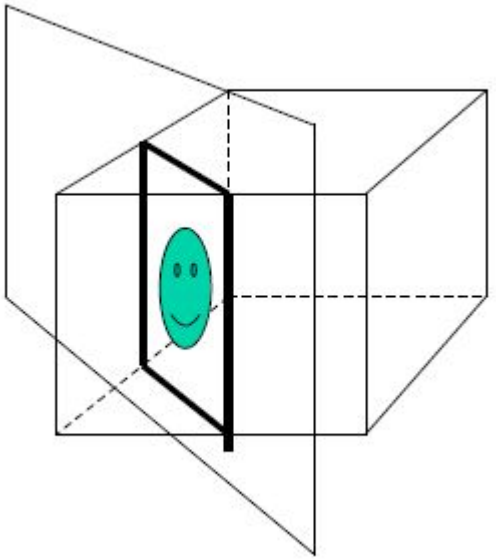
3D Texture Mapping

Texture mapping to the arbitrary slices

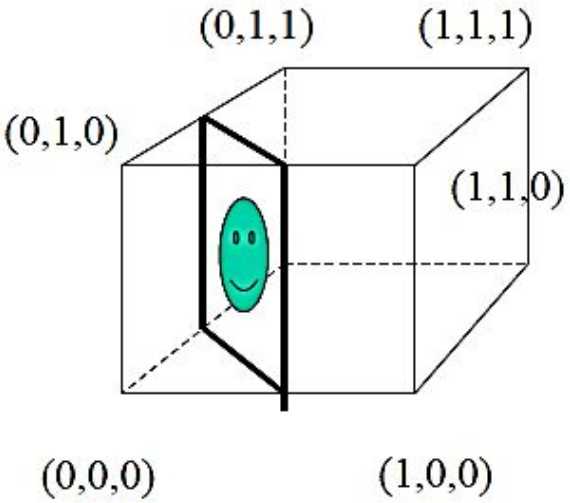
This requires 3D texture mapping hardware

Input texture: volume (pre-classified and shaded) essentially an (R,G,B,a) volume

Depending on the position of the polygon, appropriate textures are resampled, constructed and mapped to the polygon

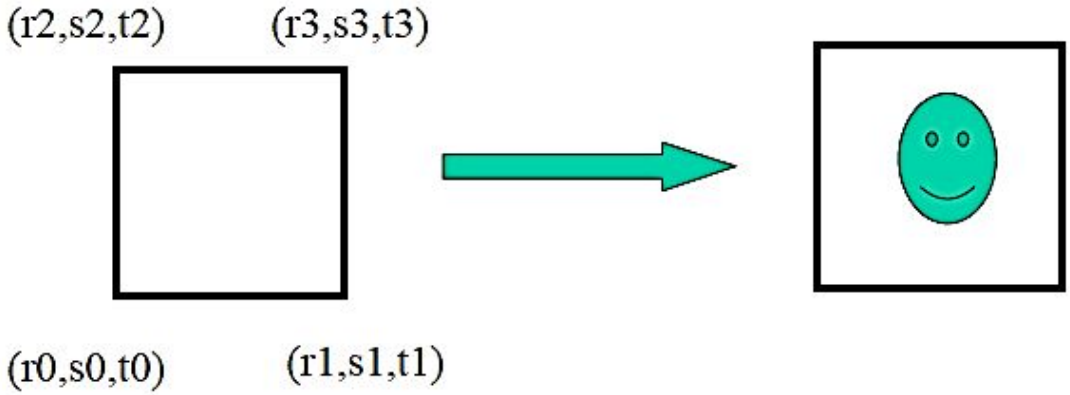


3D Texture Mapping



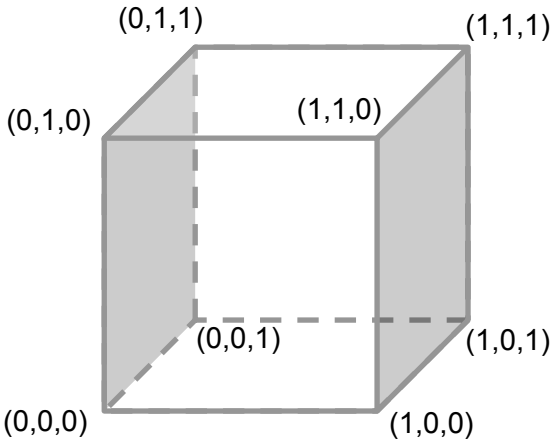
Now the input texture space is 3D

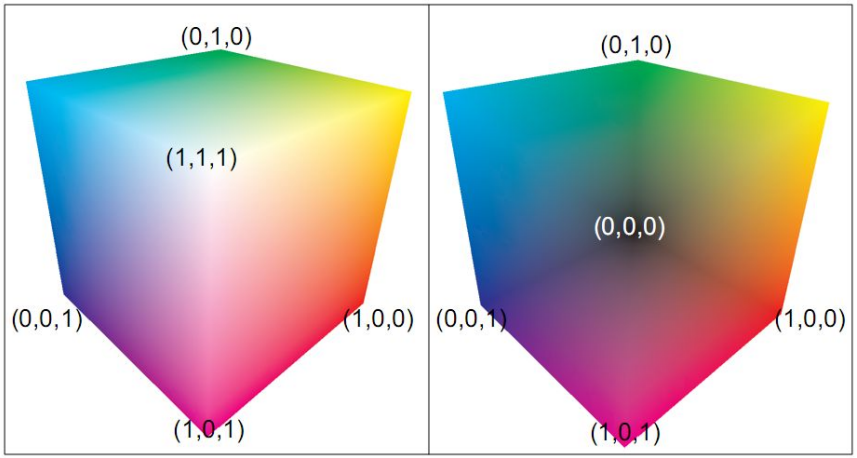
Texture coordinates: (r,s,t)



GPU ray marching

- The rays are independent
- We can process the rays in parallel
- First, we determine start and end coordinates of each ray
  - We map coordinates of box containing the data onto box geometry
  - We render the front-facing faces of the box to obtain start coordinates of a ray for each pixel on the screen
  - We render the back-facing faces of the box to obtain end coordinates of a ray for each pixel on the screen

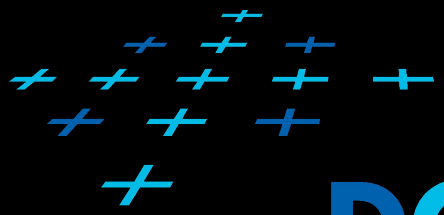




GPU ray marching

- The rays are independent
- We can process the rays in parallel
- First we determine start and end coordinates of each ray
  - We map coordinates of box containing the data onto box
  - We render the front-facing faces of the box to obtain start coordinates of a ray for each pixel on the screen
  - We render the back-facing faces of the box to obtain end coordinates of a ray for each pixel on the screen
  - We render screen aligned quad and in fragment shader construct a ray for coordinates of the fragment, integrate through the ray in for loop and return the resulting value.

Thank you for your attention



DCGI